# Closing the Security Activation Gap: A Security-by-Design Methodology for IoT Ecosystems and Its Empirical Validation in a DC Smart Microgrid

Raziel Campos-Sanchez[a,b,*], Luis Hernandez-Martinez[a], Claudia Feregrino-Uribe[b,c], Alejandro Penuelas-Angulo[d,e], Diego Cruz-Aguilar[b,c]

[a] *Electronics Department, National Institute of Astrophysics, Optics and Electronics (INAOE), Luis Enrique Erro 1, San Andrés Cholula, 72840, Puebla, Mexico*
[b] *Science and Technologies of Security Department, INAOE, Luis Enrique Erro 1, San Andrés Cholula, 72840, Puebla, Mexico*
[c] *Computational Sciences Department, INAOE, Luis Enrique Erro 1, San Andrés Cholula, 72840, Puebla, Mexico*
[d] *CEIT-Basque Research and Technology Alliance (BRTA), Manuel Lardizabal 15, Donostia/San Sebastián, 20018, Guipuzcoa, Spain*
[e] *Universidad de Navarra, Tecnum, Manuel Lardizabal 15, Donostia/San Sebastián, 20018, Guipuzcoa, Spain*

**Abstract**

A persistent security activation gap constrains IoT adoption: security mechanisms are widely available in commercial platforms yet remain inactive after standard deployment. We present a security-by-design methodology for centralized IoT ecosystems that treats identity as an intrinsic entity property, spans seven stages across four phases, and integrates cybersecurity throughout the development lifecycle. A dynamic Root of Trust process establishes verifiable cryptographic identity for each entity, providing the trusted foundation for authentication, access control, and secure communication.

We validate the methodology through a confirmatory case study applying all seven stages to a DC Smart Microgrid (SMG): two ESP32-based secondary control nodes and a Raspberry Pi 4B primary coordination agent.

*Corresponding author

*Email addresses:* rcampos@inaoe.mx (Raziel Campos-Sanchez), luish@inaoe.mx (Luis Hernandez-Martinez), cferegrino@inaoe.mx (Claudia Feregrino-Uribe), apenuelasan@ceit.es (Alejandro Penuelas-Angulo), diego.cruz@inaoe.mx (Diego Cruz-Aguilar)

Four case study questions assess guidance sufficiency, identity model unambiguity, Root of Trust verifiability, and threat coverage. All seven stages produced documented, traceable artifacts with no external guidance required; a 14-parameter identity model across four entity types was produced without ambiguity; the methodology achieves unweighted OWASP IoT Top 10 coverage of 7/10 items fully addressed and 3/10 partially addressed. The Root of Trust, which reconstructs each node's cryptographic identity from a Physically Unclonable Function response rather than a stored key and derives its session key from an ephemeral ECDH exchange for forward secrecy, was validated end-to-end on real hardware for one node across $N = 30$ hard-reset trials (30/30 success, median handshake latency 1300 ms, mean $1313 \pm 48$ ms); validating the second node remains future work. The methodology's parameterized, technology-agnostic structure is designed to generalize across diverse IoT domains; empirical validation in additional domains remains future work.

*Keywords:* Internet of Things, Security-by-Design, IoT Methodology, Root of Trust, Identity Management, Smart Microgrid, Case Study Research

---

## 1. Introduction

The Internet of Things (IoT) has evolved from a conceptual vision into a technological paradigm underpinning critical infrastructure, industrial automation, healthcare systems, and smart cities (Al-Fuqaha et al., 2015; Zhou et al., 2021). Despite this expansion, large-scale IoT adoption remains constrained by fragmented architectures, heterogeneous technologies, and—critically—the failure to integrate cybersecurity from the earliest design stages (Shahinzadeh et al., 2024; Makhdoom et al., 2018).

The consequences of this gap are well documented. The Mirai botnet, which compromised over 600,000 IoT devices to launch unprecedented distributed denial-of-service attacks, originated from misconfigured or unprotected deployments relying on default credentials (Antonakakis et al., 2017). Subsequent analyses confirm that cyberattacks on IoT infrastructure continue to escalate in frequency and sophistication, targeting energy systems, healthcare networks, and public utilities (Abraham et al., 2025; Okika et al., 2025).

A central observation motivating this work is that the principal barrier to secure IoT deployment is not the *absence* of security features but the *security activation gap*: the disconnect between security capabilities *available*

in a platform and those *operational* after a standard deployment (Figure 1). Widely adopted open-source IoT platforms provide security mechanisms that require manual post-installation configuration, effectively restricting their use to organizations with dedicated security expertise (Fagan et al., 2021).

A second, more fundamental barrier operates at the methodological level: most IoT development projects lack a structured process that integrates cybersecurity from organizational decision-making through operational maintenance. Security is typically addressed *after* architectural and technology decisions are made, producing mechanisms that are bolt-on rather than foundational, with identity management that is platform-specific rather than entity-centric.

To address both barriers, we present two complementary contributions:

1. **A security-by-design methodology** for developing centralized IoT ecosystems. The methodology treats identity as an intrinsic property of every entity, spans seven stages across four phases (organizational strategy, architecture, implementation, and operation), and embeds cybersecurity—from threat policy to Root of Trust establishment—as a foundational concern from problem definition through ongoing operation. A dynamic Root of Trust (RoT) process establishes verifiable cryptographic identity for each entity before any operational communication begins.

2. **An empirical confirmatory case study** applying the methodology to the design and implementation of a DC Smart Microgrid (SMG) IoT ecosystem, following Yin's (2014) Case Study Research design. The SMG comprises two ESP32 secondary control nodes and a Raspberry Pi 4B primary coordination agent, reaching Technology Readiness Level 3. Four case study questions evaluate guidance sufficiency, identity model unambiguity, Root of Trust verifiability, and OWASP IoT threat coverage.

**Conceptual foundation:** The identity-centric IoT paradigm, the IoT/ecosystem distinction, the formal entity identity model, and the reconceptualization of the Root of Trust as a multi-phase process are established at the conceptual and definitional level in a companion manuscript (Campos-Sanchez et al., 2026a). The present work builds directly on that foundation to specify, validate, and empirically evaluate an operative

seven-stage development methodology; it does not re-derive these definitions independently.

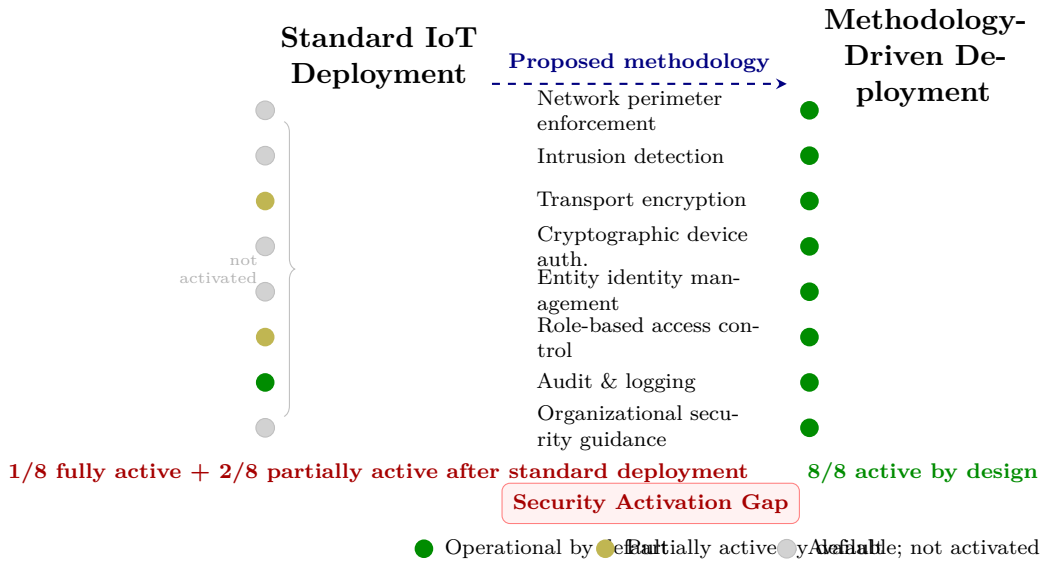



Figure 1: The security activation gap: standard IoT deployments activate only a subset of platform security controls by default; the remaining controls require manual post-installation configuration by practitioners with dedicated security expertise. The proposed methodology specifies activation of all controls by design; the TRL 3 case study reported here realizes this for most but not all (flash encryption, secure boot, and JTAG disable remain deferred to ITER 2/TRL 5+ — Limitations), consistent with an early-maturity proof of concept.

**Scope note:** The ESP32/C (ESP-IDF) firmware suite and the primary agent software implementing the SMG are presented as part of this work's own case study rather than as a companion contribution under separate review. The present work evaluates the *methodology* through the case study; implementation details are provided only to the extent required for methodological evidence.

We evaluate the methodology through: (a) stage-by-stage traceability, demonstrating that all seven stages produce documented, actionable artifacts; (b) quantitative measurements of identity provisioning and handshake latency; and (c) a systematic OWASP (Open Web Application Security Project) IoT Top 10 coverage analysis.

The remainder of this article is organized as follows. Section 2 reviews

related work. Section 3 presents the conceptual framework. Section 4 details the methodology and its STRIDE threat coverage model. Section 5 presents the SMG case study. Section 6 reports empirical results. Section 7 discusses findings, and Section 8 concludes.

## 2. Related Work

We adopt the definition of IoT methodology from Fortino et al. (2020): a structured set of guidelines, methods, practices, and documents organized within a logical framework for solving the specific problem of IoT ecosystem development. We evaluate existing methodologies against nine criteria: the eight classical dimensions drawn from the literature, supplemented by a ninth, *security validation*, which captures whether the methodology's security provisions are supported by empirical evidence from a formal case study or experiment.

Table 1 compares representative methodologies. A 2023 comparative study (Moreta et al., 2023) confirms cybersecurity integration remains the most underdeveloped dimension across IoT development methodologies: most proposals emphasize functional aspects—interoperability, scalability, technological integration—while treating cybersecurity as an afterthought or omitting it entirely (Fortino et al., 2020; Al-Fuqaha et al., 2015; Merzouk et al., 2020). Agent-based approaches (Manate et al., 2014; Ayala et al., 2015; Spanoudakis and Moraitis, 2015) provide structured development processes but lack explicit security mechanisms and identity assignment; model-driven methodologies (Morin et al., 2017; Costa et al., 2016; Cicirelli et al., 2017) address code heterogeneity and portability but not entity identity or security. A systematic review of agile IoT methodologies (Guerrero-Ulloa et al., 2023) confirms even recent structured approaches defer authentication, identity management, and threat modeling until after functional requirements are met—if addressed at all.

Security-focused proposals remain limited in scope and depth. Josyula and Gupta (2017) presents a security methodology for IoT systems engineering without specific guidelines for identity assignment or cryptographic mechanism selection. Savaglio et al. (2018) includes security aspects (authorization, key management, authentication) but not selection criteria or their timing relative to development stages. Muñoz et al. (2025) proposes a five-layer service-oriented architecture with security as a cross-cutting concern, without explaining how mechanisms are integrated or selected; its PKI-based

scheme neglects resource-constrained devices. A systematic review of IoT authentication schemes in this journal (Saqib and Moon, 2023) (over 120 proposals, 2010–2022) confirms core fragmentation: authentication mechanisms are proposed in isolation, without connection to a broader development process, formal identity model, or measurable threat coverage metric. Blanco et al. (2023) propose an ontology-driven framework for cyber-physical security requirements, providing structured elicitation but not a full lifecycle spanning organizational policy through operational deployment.

More recently, several efforts target specific IoT security domains without holistic development guidance: Zero Trust architectures for industrial IoT (Zanasi et al., 2024; Fernandez and Brazhuk, 2024), blockchain-based supply chain security (Ray et al., 2024; Fernando et al., 2023), and SDN-based IoT fog security (Javanmardi et al., 2023; Turner et al., 2023) address individual threat surfaces but not a development methodology. Visintin (2024) examines identity of things device-centrically but does not extend to ecosystem-wide identity management across heterogeneous entity types or connect identity to access control policy. PUF-based Root of Trust implementations (Sassalou et al., 2025) and hardware security modules (Bakiris and Oikonomakis, 2025) strengthen device-level identity but remain single-layer solutions, leaving the broader development process unaddressed. This work adopts an SRAM-based PUF for the ESP32, `esp32_puflib` (Stanicek, 2022), as the concrete Stage 4 technology for identity-parameter generation without persistent on-device key storage (Section 4), integrated into the full seven-stage process rather than treated as an isolated device-level mechanism.

Standards from government agencies and consortia—Secure-by-Design (UK Government Security, 2024), NIST IoT Cybersecurity Guidance (Fagan et al., 2021), ETSI EN 303 645 (European Telecommunications Standards Institute, 2020), the CSA IoT Security Controls Framework (Cloud Security Alliance, 2023), and the 2025 NIST cybersecurity supply chain risk guidance (Licata et al., 2025)—provide valuable security baselines. However, these standards remain predominantly prescriptive at the policy level, offering no technology selection criteria, no process for deriving implementation choices from organizational constraints, and no deployment automation.

Taken together, these approaches share a common limitation: the absence of a methodology that simultaneously addresses (a) organizational adoption strategy, (b) technology-agnostic architectural design with a formal entity-centric identity model, (c) a dynamic Root of Trust process establishing verifiable cryptographic identity, (d) iterative development guided by matu-

rity levels, and (e) empirical validation through a formal case study with measurable artifacts at every stage. The methodology presented in this work addresses all five dimensions. The validation follows Yin's (2014) confirmatory case study design with four research questions and documented stage-by-stage artifact traceability—we are not aware of a comparable empirical validation of a complete IoT security development methodology using this research design.

Table 1: Comparison of IoT ecosystem development methodologies (9 criteria). Tech.Dev. = technological development guidance; CyberSec. = explicit cybersecurity integration; Agnostic = technology-agnostic design; Org. = organizational aspects; Scalability; Maturity = TRL or equivalent; Interop. = interoperability; Concept. = formal conceptual framework; Sec.Val. = security provisions validated through formal case study or experiment. ✓ full, ~ partial, ✗ absent. *This work* is rated ~ on Tech.Dev. because the methodology is technology-agnostic by design: it specifies mechanism categories and stage requirements, not specific technology selection; ~ on Maturity and Scalability because the case study reaches only TRL 3 (laboratory proof of concept) on 2 nodes, not a production-scale or externally-validated deployment. Ratings for "This work" are self-assessed by the authors, not externally validated.

| **Methodology** | Tech. Dev. | Cyber Sec. | Agnostic | Org. | Scal-ability | Maturity | Inter-op. | Concept. Frame. | Sec. Val. |
|---|---|---|---|---|---|---|---|---|---|
| Manate et al. (2014) (Manate et al., 2014) | ~ | ~ | ✗ | ✗ | ✓ | ✗ | ✓ | ✗ | ✗ |
| Ayala et al. (2015) (Ayala et al., 2015) | ✓ | ✗ | ✓ | ✗ | ~ | ✗ | ✓ | ~ | ✗ |
| Spanoudakis et al. (2015) (Spanoudakis and Moraitis, 2015) | ✓ | ✗ | ✗ | ~ | ✓ | ✗ | ✓ | ✗ | ✗ |
| Patel et al. (2015) (Patel and Cassou, 2015) | ✓ | ✗ | ✗ | ✗ | ✓ | ✗ | ✓ | ✓ | ✗ |
| Costa et al. (2016) (Costa et al., 2016) | ✓ | ✗ | ✗ | ✗ | ✓ | ✗ | ✓ | ✗ | ✗ |
| Fortino et al. (2017) (Fortino et al., 2017) | ✓ | ✗ | ✗ | ✗ | ✓ | ✓ | ✓ | ✗ | ✗ |
| Josyula et al. (2017) (Josyula and Gupta, 2017) | ✗ | ~ | ✓ | ✗ | ✓ | ✗ | ✓ | ~ | ✗ |
| Savaglio et al. (2018) (Savaglio et al., 2018) | ~ | ~ | ✓ | ✗ | ✓ | ✗ | ✓ | ~ | ✗ |
| Casadei et al. (2019) (Casadei et al., 2019) | ✓ | ✗ | ~ | ✗ | ✓ | ✗ | ✓ | ~ | ✗ |
| Guerrero-Ulloa et al. (2023) (Guerrero-Ulloa et al., 2023) | ✓ | ✗ | ✓ | ~ | ~ | ✗ | ~ | ✗ | ✗ |
| Muñoz et al. (2025) (Muñoz et al., 2025) | ✓ | ~ | ✗ | ✗ | ~ | ✗ | ~ | ~ | ✗ |
| **This work (2026)** | ~ | ✓ | ✓ | ✓ | ~ | ~ | ✓ | ✓ | ✓ |

## 3. Conceptual Framework

Ambiguity in IoT definitions hinders coherent methodology design (Sorri et al., 2022). Rather than re-deriving reference definitions, this methodology builds directly on the identity-centric conceptual framework established in a companion manuscript (Campos-Sanchez et al., 2026a), which reframes IoT as a technological paradigm in which cyber-physical systems are constitutively endowed with identity, and distinguishes this paradigm from an *IoT ecosystem*—a digital environment in which entities possessing unique identities interact under a defined structure to provide services that solve identified

problems. We adopt both definitions unchanged and use them as the entry point for the present methodology.

The ecosystem comprises seven element types: IoT devices (embedded systems with ecosystem-relative identity), IoT servers (network-connected infrastructure enabling entity communication), IoT services (processes enabling ecosystem operation and security), storage systems (information persistence infrastructure), users (business-logic-level entities with defined roles), administrators (operational management entities), and communication infrastructure (data exchange enablers).

**Definition 1** (**Cybersecurity in an IoT Ecosystem**). *Intrinsic properties of the design, development, structure, integration, operation, and physical conditions that determine the vulnerability, robustness, resilience, and response to disturbances or malicious interventions in IoT ecosystems.*

We likewise adopt the reconceptualization of the Root of Trust (RoT) as a multi-phase *process*, rather than a static hardware component, established in Campos-Sanchez et al. (2026a): entities establish, exchange, and securely store identity parameters within a trusted environment before any operational communication begins. This process accommodates heterogeneous device capabilities—from constrained microcontrollers to cloud servers—supports both symmetric and asymmetric cryptographic schemes, and establishes the trusted context required for provisioning, authentication, and secure session establishment across all entity types. When Phase 1 (parameter generation) is instantiated through a Physically Unclonable Function (PUF) rather than random generation followed by persistent storage, Phase 2 (secure storage) is correspondingly trivialized: the entity's private identity parameter is never stored at rest—only non-secret helper data required to reconstruct it deterministically—removing the secure-storage attack surface rather than merely hardening it. Stage 6.1.3 (Section 4) instantiates this process concretely for the case study's entity types.

## 4. Proposed Methodology

The methodology is organized into four phases and seven stages (Figure 2). Each stage produces documented outputs that serve as inputs to the subsequent stage, and is itself decomposed into numbered sub-processes (e.g., 2.2.4, 6.1.3) that the case-study evidence cites throughout this article; the

Supplementary Material gives both the generic, technology-agnostic specification of each process (Tables S1–S4) and its instantiation for the SMG case study (Tables S5–S8).

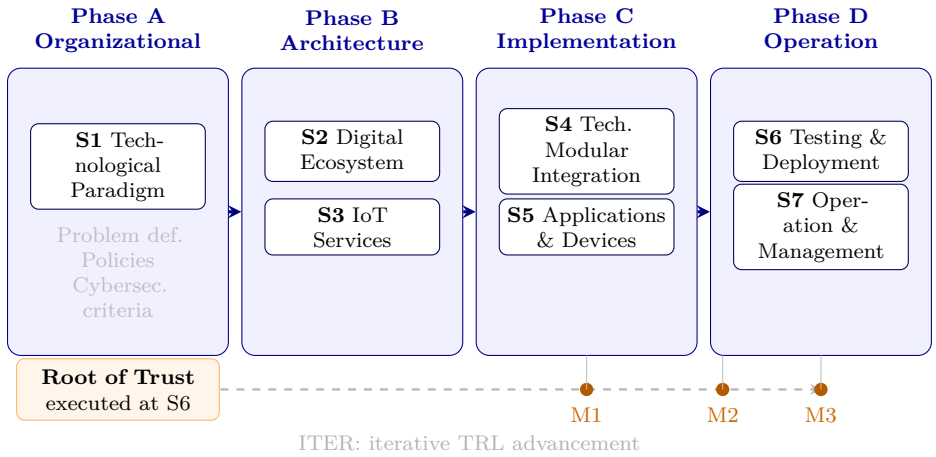



Figure 2: Organization of the proposed methodology: four phases (Organizational, Architecture, Implementation, Operation) comprising seven numbered stages (S1–S7). Iterative development cycles (ITER) advance Technology Readiness Level. Maintenance points M1 (implementation), M2 (deployment), and M3 (operation) trigger targeted stage re-entry. The Root of Trust is executed at Stage 6.

### *4.1. Phase A: Organizational (Stage 1)*

This phase addresses organizational considerations for IoT adoption. Stage 1 establishes the problem to be solved, operating policies, development policies, and cybersecurity criteria. The key processes are: problem definition with cost-benefit analysis (1.1); operating policies covering functional objectives, financial constraints, and regulatory compliance (1.2); development policies defining an iterative strategy using the TRL framework, timelines, and resource allocation (1.3); and cybersecurity policies covering criticality assessment, technological sovereignty, data sovereignty, supply chain security, and applicable government guidelines including Secure-by-Design (UK Government Security, 2024) and Privacy-by-Design (European Union Agency for Cybersecurity, 2017) principles (1.4). Processes 1.4.2–1.4.3 operationalize the organizational-strategy layer of the sovereignty framework established in Campos-Sanchez et al. (2026a), under which technological and data sovereignty are treated as emergent properties of who controls the entities' Root of Trust process rather than as external policy overlays.

### *4.2. Phase B: Architecture (Stages 2–3)*

Stage 2 defines the IoT ecosystem architecture at the entity level: integration diagram, communication topology, entity hierarchy, functional processes, data flow, and scalability strategy. Critically, Stage 2 instantiates the formal identity model established in Campos-Sanchez et al. (2026a), under which each entity $X$ in ecosystem $E$ possesses an identity:

$$I_X^E = P_{X\,pub}^E \cup P_{X\,priv}^E \tag{1}$$

where public parameters $P_{X\,pub}^E$ serve as identifiers and private parameters $P_{X\,priv}^E$ enable cryptographic operations, and authentication parameters $P_{X\,auth}^E \subset I_X^E$ and shared secrets $P_{X\,SS}^E \subset P_{X\,auth}^E$ are derived from this identity foundation (Figure 3). This methodology extends that model with an explicit cardinality count for each entity type $X$: the public-parameter and private-parameter counts are:

$$n_X^E = |P_{X\,pub}^E| \tag{2}$$

$$k_X^E = |P_{X\,priv}^E| \tag{3}$$

so that the total identity parameter count is $n_X^E + k_X^E$ and the table columns report the two component counts and their sum.

Stage 3 specifies the services required for entity operation: secure storage subsystems, access control mechanisms, identity parameter storage strategies, and regulatory compliance integration. For constrained devices, Stage 3 requires an explicit cost/risk evaluation of persistent secure-storage options (hardware security modules, encrypted flash partitions) against physically-derived alternatives that avoid persisting a secret altogether; a Physically Unclonable Function (PUF) that regenerates the private identity parameter on demand from non-secret helper data is one such alternative, and its selection at Stage 4 is a direct consequence of this Stage 3 evaluation rather than an independent design choice. Servers and users retain conventional secrets management and MFA as generic Stage-3 options; the SMG case study itself implements only a single-factor password hash for users/admins, not MFA (Table 12, Limitations).

### *4.3. Phase C: Implementation (Stages 4–5)*

Stage 4 selects enabling technologies aligned with policies from Stage 1 and architecture from Stages 2–3. Technology choices are evaluated against

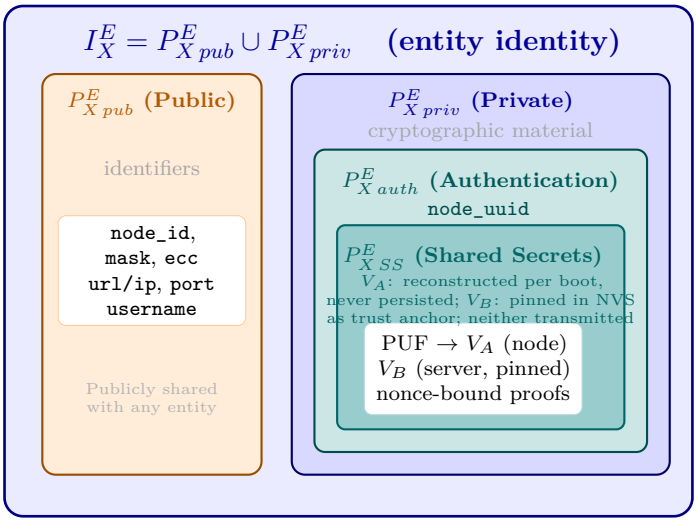



Figure 3: Identity parameter set hierarchy for entity $X$ in ecosystem $E$. Shared secrets $P_{SS}$ enable cryptographic operations; authentication parameters $P_{auth}$ prove identity; private parameters $P_{priv}$ are never disclosed; public parameters $P_{pub}$ serve as identifiers. In the SMG, the node's private identity parameter (its PUF response) lives in $P_{priv}$ but is never persisted to storage—only the non-secret mask/ECC helper data used to reconstruct it lives in ESP32 NVS, as $P_{pub}$.

maturity level (TRL), vulnerability landscape, and supply chain risk. For the SMG, Stage 4 selects `esp32_puflib` (Stanicek, 2022), an SRAM-based PUF implementation for the ESP32 family, as the identity-generation mechanism motivated by the Stage 3 secure-storage evaluation above. Because `esp32_puflib` is implemented as a native C/ESP-IDF component, Stage 4 also selects a C/ESP-IDF firmware architecture for the node, so that the PUF library is linked directly without a language-runtime wrapper; the same decision selects ESP-IDF's native `mbedtls/esp_aes` for AES-256-GCM and ECDSA support. This platform choice is itself a direct, traceable consequence of the Stage 3 evaluation, not an independent implementation preference. Stage 5 implements all ecosystem elements: servers, devices, databases, communication infrastructure, IoT services, and applications. Security is enforced through secure coding practices, unit testing of security mechanisms, and validation against applicable security guidelines (Secure-by-Design principles (UK Government Security, 2024), NIST IoT Cybersecurity Guidance (Fagan et al., 2021), and OWASP IoT Top 10).

### *4.4. Phase D: Operation (Stages 6–7)*

Stage 6 covers deployment, configuration, and testing: individual component validation, infrastructure deployment, *Root of Trust execution* for all entities (Process 6.1.3), general operation verification, and security testing (penetration testing at higher TRLs; tamper verification at TRL 3). Stage 7 addresses ongoing operation: monitoring, element maintenance, information lifecycle management, threat intelligence, element updates, incident response, and network infrastructure oversight.

### *4.5. Iterative Development and Maintenance*

The methodology supports iterative advancement through TRL-guided cycles (ITER) and three structured maintenance points: M1 (during implementation, triggers technology re-evaluation), M2 (during deployment, triggers code revision), and M3 (during operation, triggers architectural redesign). M1 fires when an implementation-phase review reveals that the chosen technology cannot satisfy a stage requirement; M2 fires when deployment-phase testing reveals that the implementation requires code-level revision to meet a security or functional requirement; M3 fires when operation-phase monitoring reveals that the deployed architecture cannot accommodate an observed operational pattern. This structure supports progressive maturation from proof-of-concept to production deployment.

### *4.6. Root of Trust Establishment*

Building on the four-phase Root of Trust process of Campos-Sanchez et al. (2026a), we instantiate its parameter generation, secure storage, exchange, and peer-management phases as concrete protocol message sequences for both cryptographic schemes.

**PUF-seeded device identity (resource-constrained devices):** Rather than generating a random device key pair and persisting the private key in flash, the device's identity key pair is derived on demand from a Physically Unclonable Function (PUF), instantiated via `esp32_puflib` (Stanicek, 2022). *Enrollment* (performed once, Table 2) reads the ESP32's RTC FAST SRAM power-up state across repeated measurements and derives two artifacts—a stable-bit mask and error-correction (helper) data—that are stored on the device and allow the same physical response to be reconstructed, with correction, on every subsequent boot. Reconstructing the response from these artifacts alone, without the chip, is intended to be infeasible, consistent with the design goal of SRAM-based PUFs (Guajardo et al., 2007); however, repetition-code helper data is known to leak some response information (Dodis et al., 2004; Maes, 2013; Delvaux et al., 2015), so this is a design property, not a proven guarantee for this construction (see Limitations). On every boot, the reconstructed response deterministically derives an elliptic-curve (P-256) key pair via HKDF-based key derivation (RFC 5869) followed by rejection sampling to a uniform scalar; the device registers its public key with the server once, at enrollment, and the corresponding private key is never transmitted, stored, or otherwise disclosed—it exists only transiently in RAM for the duration of a single boot. *Session identification* (Table 3) is asymmetric on the device side and symmetric on the server side: the device signs a server-issued nonce with its private key (ECDSA), verified against its registered public key; the server proves possession of its own pinned, per-node verifier by hashing it with a device-issued nonce, checked against the value pinned at enrollment. Each side also exchanges a fresh ephemeral public key, bound into the same proofs; the resulting ECDH secret, not the long-term verifier alone, anchors the session key (Table 3), and neither proof nor ephemeral key is reused across sessions. The device's NVS holds only the non-secret stable-bit mask/ECC data ($P_{pub}$) and the pinned per-node server verifier $V_B$; its own private key ($sk_A$, derived fresh from $\mathrm{PUF}_A$ each boot) and its ephemeral key pair exist only transiently in RAM (Section 3).

**Server identity:** the server is not a good physical candidate for PUF-based identity (no comparable unclonable substrate is assumed for the RPi4), so its identity parameter is a conventionally-provisioned verifier $V_B$, generated fresh per enrolled device and pinned on that device as its server-authentication trust anchor. $V_B$ is deliberately not shared across the fleet: recovering one device's pinned $V_B$ lets an attacker impersonate the server only to that single device, and (Table 3; Limitations) no longer by itself discloses that device's past or future P3 traffic. Server authentication is symmetric: possession of $V_B$ is exactly what the server must prove per session, via the same nonce-bound hash structure that gives the device's proof its freshness guarantee; its confidentiality on the device side is bounded by physical security (Information Disclosure, Table 6; Limitations). Device authentication, by contrast, is asymmetric: the server stores only each device's public key, so a full server compromise discloses no value sufficient to impersonate any device (Section 5.6). The two entities' parameters are thus deliberately asymmetric, bounding the single point of failure a fully-symmetric scheme would place on the server.

**Asymmetric scheme (capable devices):** Entities generate public/private key pairs, exchange public keys with identifiers, and store the peer's public key for subsequent authentication using ISO/IEC 9798-3 mutual authentication (ISO/IEC, 2019) (Table 4).

Table 2: Root of Trust process: PUF-seeded device identity establishment — Phase 1 (enrollment and registration, performed once, over an isolated/physical provisioning channel). $\text{PUF}_A$ denotes the device's PUF response, reconstructed from RTC FAST SRAM via `esp32_puflib`; $\texttt{mask}_A/\texttt{ecc}_A$ are the non-secret stable-bit mask and error-correction helper data. $(sk_A, pk_A)$ is the device's P-256 key pair, derived deterministically from $\text{PUF}_A$ together with $H_{mask/ecc}$, a hash of the stable-bit-mask/ECC helper data (Table 3, Limitations); only $pk_A$ (public) is ever registered or stored off-device. $V_B$ is a conventionally-provisioned, per-node server verifier (Section 4, "Server identity"), not PUF-derived and not shared across nodes.

| Device | Message | Server |
|---|---|---|
| $\{ID_A\}$ enroll_puf() $\rightarrow \{\text{mask}_A, \text{ecc}_A\}$ (stored, public) | | $\{ID_B, URL/IP_B, Port_B\}$ (fleet-wide, non-secret) |

*continued on next page*

*Table 2 continued from previous page*

| Device | Message | Server |
|---|---|---|
| get_puf_response() $\rightarrow$ $\text{PUF}_A$; $(sk_A,\ pk_A) \leftarrow$ HKDF-derive-keypair ($\text{PUF}_A,\ H_{mask/ecc}$) | | |
| | $\{ID_A,\ pk_A\} \rightarrow$ $\leftarrow \{ID_B,\ URL/IP_B,\ Port_B,\ V_B\}$ | generate fresh per-node $V_B$; register $\{ID_A,\ pk_A,\ V_B\}$ |
| pin (store): $\{ID_B,\ V_B\}$ as server reference | | store $\{ID_A,\ pk_A\}$ as device reference ($pk_A$ public, not a secret) |

Table 3: Root of Trust process: PUF-seeded device identity establishment — Phase 2 (per-boot reconstruction and mutual challenge; continues Table 2). Device authentication is asymmetric ($sk_A$ never transmitted, verified via $pk_A$ alone); server authentication is a nonce-bound hash-reconstruction proof over $V_B$. Each side also exchanges a fresh, session-only ephemeral public key ($pk_{eph_A}/pk_{eph_B}$), bound into both proofs via the transcript hash $T = \text{SHA-256}(\tau \parallel ID_A \parallel ID_B \parallel N_B \parallel N_A \parallel pk_{eph_A} \parallel pk_{eph_B} \parallel R_B)$ ($\tau$ a fixed domain-separation tag) against substitution or replay. $Key_{ab}$ derives from the resulting ECDH secret $Z$ together with $V_B$, giving forward secrecy (see main text, Limitations). $H_{mask/ecc} = \text{SHA-256}(\text{mask}_A \parallel \text{ecc}_A \parallel \text{mask}_A^{sleep} \parallel \text{ecc}_A^{sleep})$ is a hash of all four stable-bit-mask/ECC helper-data blobs (both the RTC and the deep-sleep-retry pair, Limitations), bound into $sk_A$'s derivation so that tampering with any of them changes the derived key unpredictably rather than leaking information through the reconstruction's error count. $\text{PUF}_A/sk_A$ and $eph_A$ are all freshly (re)generated each boot; nothing here persists between Phase 1 and Phase 2.

| Device | Message | Server |
|---|---|---|
| generate nonce $N_A$ get_puf_response() $\rightarrow$ $\text{PUF}_A$; re-derive $(sk_A,\ pk_A)$ as in Table 2; generate ephemeral $(sk_{eph_A},\ pk_{eph_A})$ | $\{ID_A,\ N_A\} \rightarrow$ $\leftarrow$ $\{ID_B,\ N_B,\ pk_{eph_B},\ R_B\}$, $R_B = \text{SHA-256}(V_B \parallel N_A \parallel pk_{eph_B})$ | generate nonce $N_B$ generate ephemeral $(sk_{eph_B},\ pk_{eph_B})$; compute $R_B$ from stored $V_B$ |

*continued on next page*

*Table 3 continued from previous page*

| Device | Message | Server |
|---|---|---|
| verify $R_B \stackrel{?}{=} \text{SHA-256}(V_B \,\|\, N_A \,\|\, pk_{eph_B})$ using pinned $V_B$ (server authentication, binds $pk_{eph_B}$); $R_A \leftarrow$ ECDSA-sign($sk_A$, $T$) | $\{pk_{eph_A},\ R_A\} \rightarrow$ | ECDSA-verify($pk_A$, $T$, $R_A$) using registered $pk_A$ (device identification, binds the full transcript $T$) |
| | $\leftarrow$ {session_ack} (or rejection if verification failed) | issue `session_ack` only if $R_A$ verifies |
| $Z \leftarrow$ ECDH($sk_{eph_A}$, $pk_{eph_B}$); SHA-256($Z \,\|\, V_B \,\|\, N_A \,\|\, N_B$) $\rightarrow Key_{ab}$ $Key_{ab}$ held only in RAM for session duration; $sk_A$ discarded with $\text{PUF}_A$, $sk_{eph_A}$ and $Z$ discarded, at end of boot/session | | $Z \leftarrow$ ECDH($sk_{eph_B}$, $pk_{eph_A}$); SHA-256($Z \,\|\, V_B \,\|\, N_A \,\|\, N_B$) $\rightarrow Key_{ab}$ $Key_{ab}$ held in RAM; $sk_{eph_B}$ and $Z$ discarded after use; $pk_A$ reference retained in server storage (public, not sensitive) |

*Implementation note:* the implementation source (`smg_puf_rot_node/`, device side; `smg_primary_server/rot_challenge_service.py`, server side; companion code repository, Materials Availability) uses `shs_b`/`V_B` for $V_B$, `challenge_a`/`challenge_b` for $N_A/N_B$, `eph_pk_a`/`eph_pk_b` for $pk_{eph_A}/pk_{eph_B}$, `server_proof`/`device_proof` for $R_B/R_A$ (`device_proof` carries a DER-encoded ECDSA signature, not a fixed-length hash), and a fixed ASCII literal (`DEVICE_PROOF_DOMAIN_TAG`, identical on both sides) for $\tau$; the four-message sequence and the $Key_{ab}$ derivation above match the implementation exactly, including concatenation order, the uncompressed-point encoding of both ephemeral public keys, and the `utils.Prehashed` convention the server uses to verify a signature computed over an already-hashed message.

### *4.7. Threat Coverage by Design: STRIDE Analysis*

We analyze how the methodology's stages address the STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and

Table 4: Root of Trust process: asymmetric key establishment.

| **Device** | **Message** | **Server** |
|---|---|---|
| $\{ID_A,\ MAC_A,$ $PubKey_A,\ PrivKey_A\}$ | | $\{ID_B,\ URL/IP_B,$ $Port_B,\ PubKey_B,$ $PrivKey_B\}$ |
| | $\{ID_A,\ MAC_A,\ PubKey_A\}$ $\rightarrow$ $\leftarrow \{ID_B,\ URL/IP_B,$ $Port_B,\ PubKey_B\}$ | |
| secure store: $\{PubKey_B,$ $ID_B,\ URL/IP_B,\ Port_B\}$ | | secure store: $\{PubKey_A,\ ID_A,\ MAC_A\}$ |

Elevation of Privilege) threat categories (Shostack, 2014) for centralized IoT ecosystems. Unlike framework-specific threat models tied to particular technologies, this analysis maps threats to methodology stages and technology-agnostic mechanism categories, ensuring applicability across implementations.

Tables 5 and 6 map each STRIDE threat to the methodology stage(s) providing countermeasures, the mechanism category mandated by those stages, and the instantiation in the SMG case study. The adversary $\mathcal{A}$ is assumed to have network access, commodity computational resources, and knowledge of the system architecture (Kerckhoffs's principle); goals include unauthorized data access, service disruption, entity impersonation, and credential exfiltration.

*continued on next page*

*Table 5 continued from previous page*

| Threat | Attack Vector | Methodology Stage(s) | Mechanism Category | SMG Instantiation | Residual Risk |
|---|---|---|---|---|---|

Table 5: STRIDE threat model: mapping to methodology stages and SMG instantiation — Part 1 (Spoofing, Tampering, Repudiation).

| Threat | Attack Vector | Methodology Stage(s) | Mechanism Category | SMG Instantiation | Residual Risk |
|---|---|---|---|---|---|
| **Spoofing** | Entity impersonation; forged credentials | 2.2.2 (auth mech.), 2.2.3 (RoT), 6.1.3 (RoT execution) | Cryptographic mutual authentication using entity-specific, PUF-derived identity commitments | PUF-derived identity (`esp32_puflib`) + asymmetric device proof/symmetric server proof; device's private key never leaves RAM; handshake before any data accepted; $N = 30$ legitimate handshakes all succeeded (Section 6.3), but FT-06's cross-chip cloning test (11/11 rejected, across four firmware generations), not a successful login, is the direct impersonation-resistance evidence | Low (requires physical device compromise) |



*continued on next page*

*Table 5 continued from previous page*

| Threat | Attack Vector | Methodology Stage(s) | Mechanism Category | SMG Instantiation | Residual Risk |
|---|---|---|---|---|---|
| **Tampering** | Data modification in transit; sensor reading manipulation | 2.2.4 (secure comms), 5.2.1 (secure dev.) | Authenticated encryption with integrity binding per message; random nonces | AES-256-GCM per message; random 12-byte nonce; GCM tag detects single-message modification | Low per-message (GCM tag); Medium for reordering/replay (no message-sequence chain, see Limitations) |
| **Repudiation** | Denial of sent data; dispute of control commands | 2.2.2 (session audit), 3.1 (data service), 7.1 (monitoring) | Session-bound data exchange with server-side session logging | Server logs each session under `node_id` and timestamp (Stage 7.1); evidentiary, not an independently-verifiable chain (trade-off, see Limitations) | Medium (logs can be disputed without independent cryptographic chain verification) |

*continued on next page*

*Table 6 continued from previous page*

| Threat | Attack Vector | Methodology Stage(s) | Mechanism Category | SMG Instantiation | Residual Risk |
|---|---|---|---|---|---|

Table 6: STRIDE threat model (continued from Table 5) — Part 2 (Information Disclosure, Denial of Service, Elevation of Privilege).

| Threat | Attack Vector | Methodology Stage(s) | Mechanism Category | SMG Instantiation | Residual Risk |
|---|---|---|---|---|---|
| **Information Disclosure** | Key extraction; credential exfiltration; traffic interception | 1.4.2–1.4.3 (sovereignty), 2.2.1 (identity params), 5.2.1 (secure dev.) | Data sovereignty policy; no persistent device-own secret to disclose; application-layer encryption | Node identity reconstructed at boot from the PUF, never persisted; only non-secret helper data and $V_B$ in NVS; AES-256-GCM keyed from $V_B$ plus a discarded per-session ECDH secret (forward secrecy) | Node identity: Low (server stores only $pk_A$). Server impersonation from a captured unit: Medium for live impersonation ($V_B$ alone suffices to complete a real-time handshake); Low for decrypting a recorded session, since $V_B$ alone no longer determines $Key_{ab}$ (see Limita- |

*Table 6 continued from previous page*

| Threat | Attack Vector | Methodology Stage(s) | Mechanism Category | SMG Instantiation | Residual Risk |
|---|---|---|---|---|---|
| **Denial of Service** | Authentication flooding; bus load disruption; node isolation | 7.1 (monitoring), 7.2.1 (anomaly detect.), 2.1.8 (hierarchy) | Anomaly-based trust scoring; degraded-mode operation; entity isolation policy | Trust score decay on repeated failures; UNTRUSTED nodes isolated (TCP dropped); last PWM duty maintained on timeout (original server modules; not yet ported to the RoT server, see Limitations) | Medium (physical-layer DoS outside software scope) |
| **Elevation of Privilege** | Node masquerading as admin; unauthorized config access | 2.1.8 (entity hierarchy), 3.2.1 (access control), 2.2.2 (auth) | Entity hierarchy with entity type determined by identification channel, not a field inside a shared message | Nodes authenticate exclusively through the P1–P3 RoT protocol; users/administrators use a separate credential-based API; P3 data-channel access is bound to the P2 session key, so a node session cannot invoke user-/admin-scoped endpoints | Low (no message format allows claiming a different entity type) |

## 5. Case Study: DC Smart Microgrid IoT Ecosystem

### *5.1. Case Study Design*

This case study follows a **confirmatory design** (Yin, 2014): the methodology constitutes the theoretical proposition; the SMG implementation constitutes the empirical test. Evidence is collected from four sources following Yin's principle of multiple evidence sources (Table 7).

Table 7: Data collection protocol: evidence sources.

| **Source** | **Evidence Type** | **Content** |
|---|---|---|
| Design documents | Documentary | Stage outputs: policies, architecture diagrams, identity tables, service specifications |
| Firmware artifacts | Physical artifact | ESP32 C/ESP-IDF modules: identity (PUF enrollment/reconstruction, `esp32_puflib`), Root of Trust handshake, secure communication (AES-256-GCM) |
| Execution logs | Archival records | Authentication logs, RoT deployment logs, functional test reports |
| Measurements | Quantitative | Identity provisioning, handshake latency, OWASP coverage |

### *5.2. Research Questions*

### *5.3. Case Context*

**Application domain:** The SMG manages distributed DC electrical energy generation, storage, and consumption across two co-located nodes in a controlled laboratory environment. The growing integration of renewable energy sources (solar panels, batteries) into local grids requires intelligent, automated management to balance generation and demand in real time without relying on centralized national grid infrastructure. This application is classified as critical infrastructure (Methodology Stage 1, Process 1.4.1), requiring the highest cybersecurity tier.

**Physical architecture:** The SMG consists of two secondary control agent nodes and one primary agent (Raspberry Pi 4B). Each secondary node connects a renewable generation element to a shared 12 V DC bus with local

Table 8: Case study research questions.

| ID | Question | Addressed by |
|---|---|---|
| CSQ1 | Does the methodology provide sufficient guidance to architect a complete, secure IoT ecosystem for a real industrial application? | Stage-by-stage traceability; completeness of outputs; functional test results |
| CSQ2 | Does the parameterized identity model enable unambiguous security mechanism definition across heterogeneous entity types? | Identity parameter table; authentication implementation; coverage across 4 entity types |
| CSQ3 | Does the Root of Trust Process establish verifiable secure communication between ecosystem entities? | RoT execution evidence; handshake latency measurements; mutual challenge-response verification |
| CSQ4 | What is the methodology's coverage of known IoT security threats relative to established standards? | OWASP IoT Top 10 mapping; security mechanism traceability |

energy storage (Table 9). The primary agent runs on a Raspberry Pi 4B connected to the DC bus as a pure load via a DC/AC inverter (12 V DC → 120 V AC) followed by a USB-C AC/DC adapter.

Table 9: Secondary node hardware components.

| **Component** | **Role** | **Specification** |
|---|---|---|
| ESP32 NodeMCU | IoT node controller | Dual-core, rated up to 240 MHz¶, 520 KB SRAM, 4 MB Flash, built-in WiFi; powered by local battery via DC-DC buck (12V→5V) |
| Epcom AGM VRLA | Local energy storage + ESP32 supply | PL-18-12, 12 V, 18 Ah; also supplies ESP32 via buck converter |
| IBFIA DC-DC SSR ×3 | Energy flow control | 3–32 VDC control, 5–60 VDC load, 40 A; driven by ESP32 LEDC on GPIO27/26/25 at 1 kHz, 10-bit |
| Schneider MCB ×3 | Overcurrent protection | 2-pole, one per SSR channel |
| Solar charge controller | Energy flow control | Commercial MPPT controller (handles droop control and battery-charging voltage regulation autonomously) |
| ACS712-05B ×2 | Current sensing | Hall-effect, 185 mV/A, 16-sample median oversampling |
| Resistive divider ×2 | Voltage sensing | 30 kΩ/7.5 kΩ (5:1 ratio); reads up to 16.5 V on 3.3 V ADC |

¶Rated maximum; the current firmware runs the CPU at 160 MHz (bench boot logs, Section 6.3), the configuration the reported latencies reflect.

**Digital architecture:** The ecosystem specifies a centralized communication topology (Stage 2.1.4) with fog processing (Stage 2.1.7): a 9-rule fuzzy EMS per ESP32 and a 27-rule security-aware EMS on the RPi4, neither yet implemented (Limitations). The entity hierarchy is: IoT Nodes (lowest privilege) < Users < Administrators. Development target: TRL 3 (proof of concept with physical elements).

### *5.4. Stage-by-Stage Traceability*

Tables 10 and 11 document the application of each methodology stage to the SMG, showing the decision made at each stage and the measurable artifact produced. This traceability table constitutes the primary evidence for CSQ1.

Table 10: Stage-by-stage traceability: methodology process → SMG decision → artifact — Part 1 (Stages 1–3).

| Stage | Methodology Process | Decision Made in SMG | Artifact |
|---|---|---|---|
| **1** Techno-logical Paradigm | 1.1 Establish problem | Manage distributed renewable energy nodes securely in a 12 V DC microgrid | Problem specification |
| | 1.1.2 Entities involved | 5 entity types: IoT nodes, server, users, admins, DC bus | Entity list |
| | 1.4.1 Criticality | Classified as critical infrastructure (electrical energy) | High-security cybersecurity policy |
| | 1.4.2 Technological sovereignty | Self-hosted; no third-party cloud; ESP32 + local RPi4 server | Architecture constraint record |
| | *Of the seven conceptual-framework element types (Section 3), the SMG instantiates five: storage/communication infrastructure are subsumed into the server, and IoT services into the device/server software. Of the five entity types, four (nodes, server, users, admins) carry digital identity (Table 12); the DC bus is tracked via P3 sensor-data integrity instead.* | | |
| **2** Digital Ecosystem | 2.1.4 Communication topology | Centralized: all nodes report to primary agent | Architecture diagram |
| | 2.1.7 Processing topology | Fog: local EMS on ESP32; global coordination on RPi4 | Processing topology record |
| | 2.2.1 Identity definition | 4 entity types, 14 identity parameters total (non-uniform per type; Table 12) | Identity parameter table |

*continued on next page*

*Table 10 continued from previous page*

| Stage | Methodology Process | Decision Made in SMG | Artifact |
|---|---|---|---|
| | 2.2.2 Authentication mechanism | Asymmetric (ECDSA) PUF-seeded device proof + symmetric, conventionally-provisioned per-node server proof; AES-256-GCM session encryption | Authentication diagram |
| | 2.2.3 Root of Trust process | 4-phase RoT designed for node–server pairs | RoT execution log |
| **3** IoT Services | 3.1 Service definition | 4 services specified: data ingestion, EMS coordination, auth API, monitoring; only RoT-based node authentication is currently implemented (Limitations) | Service specification |
| | 3.2.1 Access control | Entity type determined by identification channel (Section 5.6), not a token field; a separate token-based admin/user API is specified but not yet ported (Limitations) | Access control matrix |

Table 11: Stage-by-stage traceability (continued from Table 10) — Part 2 (Stages 4–7).

| Stage | Methodology Process | Decision Made in SMG | Artifact |
|---|---|---|---|
| **4** Tech. Modular Integration | 4.1.1 TRL planning | TRL 3 target for ITER 1; lab. proof of concept | TRL milestone record |

*continued on next page*

*Table 11 continued from previous page*

| Stage | Methodology Process | Decision Made in SMG | Artifact |
|---|---|---|---|
| | 4.1.2 Platform selection | ESP32 + C/ESP-IDF: WiFi, SHA-256 HW, NVS, `esp32_puflib` as native component | Technology selection rationale |
| | 4.1.3 Communication | WiFi IEEE 802.11 b/g/n + TCP; application-layer mutual-authentication protocol (`rot_session.c`/ `rot_challenge_ service.py`) | Protocol selection record |
| | 4.2 Vulnerability mitigation | ESP-IDF v5.4.2 pinned (per the flashed firmware build log); a per-session random nonce for every P2/P3 message (no nonce reuse) | Risk register |
| **5** Applications & Devices | 5.1.1 Server | `rot_challenge_service.py` on RPi4 (raw TCP/JSON); REST API/time-series DB/dashboard specified but not yet ported (Limitations) | Server deployment |
| | 5.1.2 Devices | ESP32 C/ESP-IDF firmware: identity (PUF), Root of Trust, secure-communication modules; sensor/EMS not yet implemented (see Limitations) | Firmware artifact |
| | 5.2.1 Secure development | NVS stores only non-secret mask/ECC helper data, plus `id_b`/$V_B$/WiFi SSID-password (no device-private secret persisted); `esp_fill_random()` nonces; no credential value in firmware source or binary | Code review record |

*continued on next page*

*Table 11 continued from previous page*

| Stage | Methodology Process | Decision Made in SMG | Artifact |
|---|---|---|---|
| **6** Testing & Deployment | 6.1.3 RoT execution | Validated end-to-end on 1/2 nodes (NODE_01; NODE_02 pending), $N = 30$ trials, 30/30 success (Section 6.3) | PUF bench (Section 6.3) |
| | 6.2.1 Functional testing | Boot, PUF reconstruction, mutual authentication (P2), encrypted exchange (P3) all pass across $N = 30$ trials; sensor/EMS tests not yet applicable | PUF bench (Section 6.3) |
| | 6.2.2 Tamper verification | NVS tampering confirmed as auth. failure (single negative test; full pen. testing deferred to ITER 2) | Serial boot log (not separately archived; see FT-06 for a deposited negative-test artifact of the same kind) |
| **7** Operation & Mgt. | 7.1.1 Monitoring | Authentication status and session history from server-side logs; V/I/P/PWM telemetry is specified but not yet implemented (no sensor module, Limitations) | Dashboard artifact |
| | 7.2.2 Update elements | OTA update policy defined with authentication requirement (ITER 2 target) | Update policy |

### *5.5. Identity Parameter Definition (Stage 2.2.1)*

Applying Equations (1)–(3) to the SMG entities produces a 14-parameter identity model (Table 12). The public/private/authentication parameter separation directly determined the authentication mechanism (PUF-derived commitment using the private, non-persistent parameter) and the secure storage requirement for the IoT Node: none, beyond the non-secret mask/ECC helper data already classified as public.

The entity-specific separation of identity parameters produced a direct, unambiguous mapping to implementation artifacts: $\texttt{mask}_A$ and $\texttt{ecc}_A$ correspond to distinct NVS keys written at enrollment time; no parameter redef-

Table 12: Identity parameter table: Stage 2.2.1 output for the SMG. $\texttt{mask}_A/\texttt{ecc}_A$: non-secret stable-bit mask and error-correction helper data generated at enrollment by `esp32_puflib`, stored in ESP32 NVS. PUF response$^{\dagger}$: the device's physically-derived private identity parameter, reconstructed from $\texttt{mask}_A/\texttt{ecc}_A$ on every boot and never persisted (see note below). $V_B^{\ddagger}$: the server's per-node authentication verifier (see note below). `id_a`$^{\S}$: the device's wire identifier, currently a TRL 3 implementation gap (see note below). Following Eqs. (2)–(3), $n = n_X^E$ is the public-parameter count, $k = k_X^E$ the private-parameter count, and Total $= n_X^E + k_X^E$ the total identity-parameter count.

| **Entity Type** | **Class** | $P_{pub}$ | $P_{priv}$ | $P_{auth}$ | $n$ | $k$ | **Total** |
|---|---|---|---|---|---|---|---|
| IoT Node (ESP32) | Device | `id_a`$^{\S}$, $\texttt{mask}_A$, $\texttt{ecc}_A$ | PUF response$^{\dagger}$ | PUF response$^{\dagger}$ | 3 | 1 | 4 |
| IoT Server (RPi4) | Server | `url/ip`, `port`, `id_b` | $V_B^{\ddagger}$ | $V_B^{\ddagger}$ | 3 | 1 | 4 |
| User | User | `username` | `user_uuid`, `password` | `user_uuid`, `password` | 1 | 2 | 3 |
| Administrator | Admin | `username` | `admin_uuid`, `password` | `admin_uuid`, `password` | 1 | 2 | 3 |
| **Total** | | **8** | **6** | **6** | | | **14** |

inition was required at the specification level for those two. `id_a` does not yet follow the same path — it is a compile-time placeholder, not an NVS-provisioned per-device value (Table 12, note $^{\S}$) — which the specification calls for but the TRL 3 implementation has not yet delivered (see Limitations). In the present case study, $P_{auth}$ and $P_{priv}$ coincide for every entity type (all private parameters participate in authentication, as shown in Table 12); this is a case-study-specific coincidence, not a general property of the model—Campos-Sanchez et al. (2026a) define only the broader relation $P_{auth} \subset I$ (Figure 3), so future case studies may include a private parameter not used for authentication (e.g., a long-term audit key), in which case $P_{auth} \subsetneq P_{priv}$ rather than $P_{auth} = P_{priv}$.

$^{\dagger}$**Note on the PUF response as a non-persistent private parameter:** the formal identity model $I_X^E = P_{X\,pub}^E \cup P_{X\,priv}^E$ (Section 4) classifies a parameter as private or public by its confidentiality role, not by whether it is stored. The IoT Node's PUF response satisfies $P_{priv}$'s role—it is never disclosed and underlies the device's authentication proof—while never existing as data at rest: it is regenerated from $\texttt{mask}_A/\texttt{ecc}_A$ (both $P_{pub}$) on every boot. This is the concrete instance of the Phase 2 (secure storage) trivialization

noted in Section 3.

[‡]**Note on $V_B$ as the server's private/authentication parameter:** the server's row describes its own proof of identity to a device, made by demonstrating knowledge of $V_B$ (Table 3) — the server-side analogue of the Node's PUF response. `id_b` (fleet-wide, non-secret) sits under $P_{pub}$ with `url/ip` and `port`. The device separately pins its own copy of $V_B$ as a trust anchor (Section 4, "Server identity"); this is a stored reference to *another* entity's parameter, not an additional Node parameter, so it is not double-counted.

[§]**Note on `id_a`:** unlike `id_b` (freshly generated per deployment), `id_a` is currently a compile-time placeholder shared by every unit, not a per-device NVS value; it identifies the device on the wire but is not part of the authentication proof (see Limitations for the implications).

### *5.6. Security Mechanism Analysis: STRIDE–to–Protocol Mapping*

The methodology-level STRIDE analysis (Section 4.7) identifies mechanism categories that address each threat category at the framework level, independent of implementation technology. This subsection demonstrates how the SMG's Stage 4 technology selection instantiates those mechanisms through three authentication and data-exchange protocol phases (P1–P3), establishing a complete traceability chain from threat identification to deployed countermeasure.

**Scope note:** Phases P1–P3 constitute an *implementation-specific* Stage 4 technology choice for the SMG (see Introduction scope note). Any protocol providing equivalent mechanism categories—pre-deployment identity isolation (P1), nonce-bound mutual identification without key transmission (P2), and per-message authenticated encryption (P3)—would satisfy the same STRIDE and OWASP properties against the same methodology requirements. This design intentionally omits a cryptographic session-chain/audit mechanism; Tables 5/6 and the Limitations (Section 7) record the resulting Repudiation/I8 trade-off explicitly.

Table 13 describes each phase's security role, mechanism, and STRIDE/OWASP coverage. Table 14 summarizes the STRIDE coverage matrix across phases.

Table 13: SMG authentication protocol phases: security role, mechanism, and coverage mapping (P1–P3). STRIDE: S = Spoofing, T = Tampering, R = Repudiation, ID = Information Disclosure, DoS = Denial of Service, EoP = Elevation of Privilege.

| **Ph.** | **Name** | **Security Role** | **Mechanism** | **STRIDE** | **OWASP** |
|---|---|---|---|---|---|
| P1 | Enrollment | Identity establishment before any network exposure | `enroll_puf()` (`esp32_puflib` (Stanicek, 2022)): 20 RTC-FAST-SRAM measurements → stable-bit mask + 8×-repetition-code ECC helper data (both non-secret), written to ESP32 NVS via USB serial on isolated workstation; WiFi SSID/password are provisioned into the same NVS image out-of-band (`tools/provision_server_ref.py`), never compiled into firmware source (`id_a` remains a compile-time constant, see Limitations); device public key $pk_A$ registered with the server, which generates and pins a fresh per-node verifier $V_B$ in return; no raw PUF response or private key is ever written to flash (a library defect that briefly violated this was found and fixed during this work, see Limitations) | S, ID | I1, I9 |

*continued on next page*

*Table 13 continued from previous page*

| Ph. | Name | Security Role | Mechanism | STRIDE | OWASP |
|---|---|---|---|---|---|
| P2 | Root of Trust | Per-boot response reconstruction and nonce-bound mutual identification without key transmission; ephemeral ECDH session key (forward secrecy) | `get_puf_response()` reconstructs $\text{PUF}_A$ (see Section 6.3 for the measured byte length, which varies slightly by enrollment); $(sk_A, pk_A)$ re-derived as in Table 2, now also bound to a hash of the stable-bit mask/ECC helper data (Limitations); device proves possession of $sk_A$ via ECDSA signature over a server-issued nonce and both ephemeral public keys; server proves possession of this node's $V_B$ via a nonce-bound hash (Table 3); $Key_{ab} \leftarrow$ SHA-256$(Z \,\|\, V_B \,\|\, N_A \,\|\, N_B)$, $Z$ a discarded ECDH secret; none of the raw response, $sk_A$, $V_B$, or $Z$ is ever transmitted; validated end-to-end on real hardware (Section 6.3) | S | I1, I3 |
| P3 | Data Exchange | Per-message authenticated encryption | $c_i \leftarrow$ AES-256-GCM$(Key_{ab}, \text{nonce}_i, m_i)$ via ESP-IDF `mbedtls/esp_aes`; a fresh random 12-byte nonce per message; the GCM authentication tag detects any single-message modification | S, T, ID | I2, I3, I6, I7 |

**Spoofing** is countered at two layers: P1 ties valid identity to physical access to the specific enrolled chip rather than a copyable stored secret; P2's nonce-bound challenge requires fresh, per-session proof from each side—an ECDSA signature from the device, a hash-reconstruction proof over $V_B$ from the

Table 14: STRIDE coverage matrix across protocol phases. ✓ = primary countermeasure (first line of defense); ~ = secondary/partial contribution; — = not applicable.

| **STRIDE Threat** | **P1** | **P2** | **P3** |
|---|---|---|---|
| Spoofing | ✓ | ✓ | — |
| Tampering | — | ~ | ✓ |
| Repudiation | — | — | ~ |
| Info. Disclosure | ✓ | ✓ | ✓ |
| Denial of Service | — | ~ | — |
| Elev. of Privilege | ✓ | ✓ | — |

server—without revealing the physical response, $sk_A$, or $V_B$ itself, confirmed bidirectionally before any session key is derived.

**Tampering** is detected at message creation time by the AES-256-GCM authentication tag on every message (P3); P2's mutual identification additionally binds the session key to both parties' proven identities. The scheme has no session-level hash chain, so retrospective re-verification of message ordering is unavailable (downgraded to ~ in Table 14; see Limitations).

**Repudiation** coverage is evidentiary rather than cryptographic: P3 messages are authenticated in transit and the server logs each session (`node_id`, timestamp, Stage 7.1), but no independently-verifiable chain binds messages to a session without trusting the server's logs, a deliberate trade-off for a smaller on-device footprint (see Limitations).

**Information Disclosure** is addressed at the source (P1: no key ever exists in storage, only non-secret helper data), during derivation (P2: the physical response is reconstructed transiently in RAM and never transmitted; neither $sk_A$ nor $V_B$ is ever sent, only proofs over them), and in transit (P3: per-message AES-256-GCM under a key that also depends on a discarded per-session ECDH secret, with random nonces).

**Denial of Service** is partially addressed at P2: the server's first reply ($R_B$) is one nonce plus one SHA-256 hash, cheap regardless of the requester's authenticity, but no further session state is committed—and no `session_ack` issued—until the device's ECDSA proof $R_A$ verifies, imposing an asymmetric cost on an attacker holding a session open. Volumetric and application-layer flooding beyond the identification gate remain outside P1–P3's scope; Stage 7

trust-based isolation (Table 6) is the mandated mechanism, implemented in the SMG's original server modules but not yet ported to the current RoT server (see Limitations).

**Elevation of Privilege** is prevented at P1, since entity type is determined by which identification channel a party uses, not a field inside a shared message: nodes authenticate exclusively through the P1–P3 RoT protocol, while users and administrators use a separate, credential-based API (Table 12); no message in Tables 2/3 carries an entity-type field. P3 API access is bound to the P2 session key, so a node session cannot invoke user- or admin-scoped endpoints.

## 6. Empirical Evaluation

### *6.1. CSQ1: Architecture Guidance Sufficiency*

**Traceability completeness:** The stage-by-stage traceability tables (Tables 10 and 11) indicate that all 7 stages produced documented, verifiable artifacts—23 distinct process-to-artifact rows across the four phases. No stage required design decisions beyond what the methodology's processes prescribe.

**Functional validation:** Table 15 reports the functional test results, drawn from the $N = 30$ real hard-reset bench trial (Section 6.3) and the dedicated tamper-verification test.

Table 15: Functional test results (Stage 6.2.1). FT-01–FT-04 pass rates are over the $N = 30$ hard-reset bench trial (Section 6.3); FT-05 is a single dedicated negative test; FT-06 is a cross-chip cloning negative test repeated 11 times across four firmware generations (Limitations). Sensor/EMS functional tests are not yet applicable, since those modules are not implemented in the current firmware (see Limitations).

| ID | Description | Result |
|---|---|---|
| FT-01 | Boot: PUF identity reconstructed from NVS helper data (mask/ECC); no raw key or PUF response in NVS | Pass (30/30) |
| FT-02 | WiFi connect: associates to the configured access point and obtains an IP address | Pass (30/30) |

*continued on next page*

*Table 15 continued from previous page*

| ID | Description | Result |
|---|---|---|
| FT-03 | P2 Root of Trust: asymmetric device proof (ECDSA) + symmetric server proof both verify, over an ephemeral ECDH exchange; mutual authentication established | Pass (30/30) |
| FT-04 | P3 session: AES-256-GCM encrypted demo message sent and acknowledged | Pass (30/30) |
| FT-05 | Tamper verification: deliberate NVS parameter corruption produces authentication failure (single targeted negative test, log not separately archived) | Pass (1/1) |
| FT-06 | Cross-chip cloning: SMG_NODE_01's entire NVS (PUF helper data, server reference) copied onto SMG_NODE_02's physical chip; server rejects the resulting device proof (signature invalid). Repeated 11 times across four firmware generations (1 original, 4 error-rate-relaxed, 3 first-generation HDM binding, 3 extended HDM binding), all rejected (Limitations) | Pass (11/11) |
| | **Overall pass rate** | **6/6 tests passed = 100%** |

**Iterative development confirmation:** At least six maintenance moments were identified and resolved during real-hardware bench validation and subsequent review passes on SMG_NODE_01 (Section 6.3, Limitations): an enrollment-state-confusion bug causing an unbounded re-enrollment loop; the session-key forward-secrecy fix (ephemeral ECDH); moving the WiFi credential from a firmware constant to NVS provisioning; erasing the `PUF_FREQUENCY` blob left in NVS after enrollment; relaxing the miscalibrated local reliability gate; and the helper-data-manipulation mitigation (including extending its scope after an initial gap was found). Each was a code-level revision triggered by testing or review, not an architectural redesign. This confirms the methodology's M2 maintenance pathway operates as specified at TRL 3 (see Discussion for the M1/M2 classification and further incidents pending SMG_NODE_02 validation).

**Answer to CSQ1:** The methodology provided sufficient guidance to produce a complete, operational IoT ecosystem. All 7 stages produced documented outputs with no external guidance required.

### *6.2. CSQ2: Identity Model Unambiguity*

Applying the identity model equations to 4 entity types produced 14 distinct parameters with zero overlap between entity classes (Table 12). The public/private/authentication tripartite structure directly determined: (a) what is public non-secret helper data stored in NVS ($\texttt{mask}_A$, $\texttt{ecc}_A$), (b) what is never persisted and is instead reconstructed on every boot (the PUF response), and (c) what constitutes the device's authentication proof (the reconstructed PUF response alone; `id_a` identifies the device on the wire but is not itself cryptographically verified, and is not yet per-device provisioned, see Limitations). Table 16 summarizes these identity model metrics; of the SMG's 2 nodes, 1 (SMG_NODE_01) has been provisioned and validated under this model so far, with SMG_NODE_02 pending (Section 6.3).

Table 16: CSQ2 identity model metrics.

| **Metric** | **Value** |
|---|---|
| Entity types defined | 4 |
| Total identity parameters | 14 |
| Public parameters ($P_{pub}$) | 8 |
| Private parameters ($P_{priv}$) | 6 |
| Authentication parameters ($P_{auth}$) | 6 |
| Nodes provisioned via RoT | 1/2 |
| Parameter collisions (overlapping definitions) | 0 |
| Additional design decisions required at Stage 5 | 0 |

**Answer to CSQ2:** The identity model is sufficient and unambiguous. All 4 entity types received complete, non-overlapping identity definitions that translated directly to implementation artifacts without modification.

### *6.3. CSQ3: Root of Trust Verifiability*

The RoT process was executed and bench-validated end-to-end on real hardware for SMG_NODE_01 (ESP32-D0WD-V3 rev 3.1), connected over the real `SMG_Primary` WiFi access point to the real RPi4 primary agent (`rpi4-server`) running the server-side counterpart

`rot_challenge_service.py`. **Validation to date covers one of the SMG's two nodes; SMG_NODE_02 validation remains future work.** Post-provisioning, node authentication was verified by deliberate NVS parameter tampering—corrupted parameters consistently produced authentication failures, confirming that the RoT creates a verifiable security boundary.

Table 17 reports results from $N = 30$ successive hard-reset trials against a single, fixed enrollment (the stable-bit mask and ECC helper data were generated once via `enroll_puf()` and not regenerated between trials); each trial therefore measures per-boot reconstruction-and-handshake reliability, not enrollment-to-enrollment variability, and exercises Phases P2 and P3 of the protocol (Section 5.6) against the real server on every reset, without repeating P1 enrollment.

Table 17: PUF-based RoT real-hardware bench trial: SMG_NODE_01, $N = 30$ successive hard-reset trials against one fixed enrollment. `esp32_puflib`'s own instrumentation reports two per-attempt figures: hamming-weight, a hard gate on the raw measurement (must exceed 48.5%, a randomness sanity check unrelated to any specific enrollment), and raw bit-error rate, reported here for reference only — it does not gate acceptance (Limitations). The latter counts only the bits the ECC decoder corrects within the stable-bit-masked region (422×64 = 27,008 bits for this enrollment), but divides by the full 32768-bit measured region rather than the masked subset, so it understates the error rate over the stable bits actually used; the per-stable-bit figure is proportionally higher by a factor of 32768/27008 ≈ 1.21. The server's ECDSA verification, not either local figure, is what establishes identity.

| **Metric** | **Value** ($N = 30$) | **Notes** |
|---|---|---|
| End-to-end success rate | 30/30 (100%; 95% Clopper–Pearson lower bound 88.4%) | Mutual identification (P2, with ephemeral ECDH) + AES-256-GCM demo message exchange (P3), confirmed on both device and server logs; repeated trials on one device/enrollment, not independent samples (see discussion below) |

*continued on next page*

*Table 17 continued from previous page*

| **Metric** | **Value** ($N = 30$) | **Notes** |
|---|---|---|
| Reconstruction attempts per trial | 1/1 in every trial (30/30 succeeded on attempt 1) | No retry triggered; the library's two-tier retry mechanism remains available as a fallback but was not exercised in this round (see discussion below) |
| Attempt-1 raw bit-error rate | $0.397 \pm 0.052\,\%$ (range 0.308–0.485 %) | Diagnostic only (see discussion below); above the library's original 0.15% figure in every trial, same as every prior round, yet every reconstruction was accepted on the first attempt and verified by the server |
| Handshake latency, first exchange (connect → server authenticated) | $680.0 \pm 9.3$ ms | Server proof over $V_B$, binds the server's ephemeral public key; no ECDSA involved |
| Handshake latency, second exchange (server authenticated → device authenticated) | $359.7 \pm 46.7$ ms (median 345) | Device's ECDSA sign plus the server's verify, over the full transcript including both ephemeral keys; now also reads all four stable-bit mask/ECC helper-data blobs (RTC and deep-sleep-retry pairs) and hashes them for the extended HDM-mitigation key derivation (Limitations) |
| Handshake latency, key derivation (device authenticated → session established) | $273.7 \pm 4.8$ ms | ECDH shared-secret computation plus final SHA-256 derivation of $Key_{ab}$ |

*continued on next page*

*Table 17 continued from previous page*

| **Metric** | **Value** ($N = 30$) | **Notes** |
|---|---|---|
| Handshake latency, total (TCP connect → session established) | median 1300 ms (mean $1313.3 \pm 47.6$ ms; range 1290–1520) | About 50 ms slower by mean than the previous (first-generation, two-blob) HDM-mitigation round (median 1250 ms, mean $1262.7 \pm 47.9$ ms), consistent with reading and hashing roughly 7.5 KB more NVS data for the two additional helper-data blobs; about 245 ms slower by mean than the pre-mitigation round (median 1060 ms, mean $1067.7 \pm 23.6$ ms) (see discussion below) |
| WiFi association latency | $3545.3 \pm 20.8$ ms (range 3520–3640) | ESP-IDF stack retried the first association attempt in all 30 trials; a WiFi-stack characteristic, not a RoT protocol property |
| Device key-pair ($pk_A$) reproducibility | Consistent with a single registered value in 30/30 trials | All 30 trials authenticated against the one server-registered public key; confirmed indirectly, since the raw key material is not logged |

Four findings stand out, the first two of which were corrections made during this validation rather than pre-existing design properties. First, an earlier version of this round (`bench_results_20261015/`) exposed a local reliability gate that rejected every single reconstruction attempt (0/30 end-to-end success, all 30 trials timing out): across the 180 first-attempt measurements logged, raw bit-error rate ranged 0.455–0.760% (mean 0.589%), and across the 150 retry-path measurements, 0.226–0.406% (mean 0.329%) — both consistently above the library's original 0.15% figure, so

every attempt, on either path, was rejected before a handshake was even attempted. On inspection, this error-rate gate turned out not to be a security boundary: `correct_data()` already runs the measurement through an 8-bit majority-vote repetition code regardless of the error count, producing its best-effort reconstruction either way, and a wrong reconstruction simply fails the server's ECDSA verification downstream, exactly like any other incorrect answer — the same property Table 17's caption and the Limitations already identify as the actual security boundary. The error-rate gate was therefore relaxed to a diagnostic (Limitations), keeping only the hamming-weight sanity check as a hard gate. Second, the same review pass found that `PUF_FREQUENCY`, a raw per-bit measurement blob, was never erased from NVS after enrollment, and that `PUF_MASK`/`ECC_DATA` were bound into key derivation only via the physical PUF response, not against tampering with the NVS copies themselves (Limitations); both were fixed, which changes the derived key pair, so SMG_NODE_01 was re-enrolled under the fully corrected firmware and a round (`bench_results_20261015_hdm/`, now superseded) was collected against that new identity: 30/30 success, every trial succeeding on the first attempt. A second, independent review pass then found that binding was still incomplete: it covered only the first-attempt/RTC helper-data pair (`PUF_MASK`/`ECC_DATA`), leaving the deep-sleep-retry pair (`PUF_SLEEP_MASK`/`ECC_SLEEP_DATA`) unbound and the same oracle re-exploitable via that path; the hash was extended to cover all four blobs, requiring a third re-enrollment, and the round reported here (`bench_results_20261016_hdm2/`) was collected against that twice-corrected identity: 30/30 success, every trial succeeding on the first attempt, with the library's two-tier retry mechanism never triggered. That the retry-path blobs are genuinely bound in (not just read without effect) was confirmed directly: flashing SMG_NODE_01's NVS with a single bit flipped in either `PUF_SLEEP_MASK` or `ECC_SLEEP_DATA` (while leaving the other three blobs, and the physical PUF itself, untouched) caused the server to reject the device's signature both times, even though first-attempt reconstruction — which never reads the retry-path blobs — continued to report a normal error rate; restoring the unmodified image immediately re-established a successful handshake, isolating the retry-path blobs as the only changed input. Third, the end-to-end protocol—PUF reconstruction, nonce-bound mutual identification in both directions, and AES-256-GCM data exchange—succeeded in every trial, confirming a stable identity (and key pair) despite per-attempt measurement noise that now only reaches the

server's cryptographic check. Fourth, the handshake latency splits into three phases, detailed in Table 17: server authentication, the device's ECDSA sign/verify, and ECDH key derivation; total latency is tightly clustered (median 1300 ms, mean $1313.3 \pm 47.6$ ms) but about 50 ms slower by mean than the first-generation (two-blob) HDM round (median 1250 ms, mean $1262.7 \pm 47.9$ ms) and about 245 ms slower by mean than the pre-mitigation round (median 1060 ms, mean $1067.7 \pm 23.6$ ms) — consistent with, though not proof of, the additional NVS reads and hashing the extended binding performs, concentrated mainly in the second phase (Table 17). The server log for this round shows 33 "handshake OK" entries in total: 30 from the reported trials, one incidental verification handshake performed immediately after re-provisioning, one from the single-bit-flip tamper test's final restore-and-confirm step, and one from SMG_NODE_01 itself reconnecting when its NVS was briefly restored during the FT-06 re-run below; the same log's 5 "FAILED" entries split between the two single-bit-flip tamper trials just described and the 3 FT-06 trials below. The 30 trials are repeated measures on one device/enrollment, not independent samples (Limitations). Raw logs and summaries for all six bench rounds are in the companion artifact repository under `case study SMG/Implementation/smg_puf_rot_node/` (Materials Availability): `bench_results_20261016_hdm2/` is the round reported here; `bench_results_20261015_hdm/` (30/30, first-generation two-blob HDM binding; mean $1262.7 \pm 47.9$ ms), `bench_results_20261015_fixed/` (30/30, same relaxed error-rate gate but the pre-HDM-mitigation identity; mean $1067.7 \pm 23.6$ ms), `bench_results_20261015/` (0/30, the miscalibrated-gate round above), `bench_results_20261013/` (30/30, forward-secrecy added but the retry path's error-rate gate not yet enforced; mean $1131 \pm 50$ ms), and `bench_results_20261005/` (pre-forward-secrecy protocol; mean $558.0 \pm 181.2$ ms) are all superseded and kept for provenance. All reported standard deviations in this paper are population (not sample) statistics.

**Answer to CSQ3:** The RoT process established verifiable mutual authentication for SMG_NODE_01 across $N = 30$ successive hard-reset trials (30/30 end-to-end success, Table 17): the device authenticated the server via a nonce-bound hash-reconstruction proof over a per-node verifier, and the server authenticated the device via an ECDSA signature verified against the device's registered public key, with neither the device's private key, the server's per-node verifier, nor the ECDH shared secret ever transmitted. Median handshake latency was 1300 ms (mean $1313.3 \pm 47.6$ ms), splitting across server authentication (680.0 ms), the device's ECDSA sign/verify

(359.7 ms), and ECDH key derivation (273.7 ms, Table 17) — the last two phases carry the direct cost of the forward-secrecy and helper-data-manipulation fixes (Limitations) and did not exist in the original protocol. Validating SMG_NODE_02 under this scheme remains future work.

### *6.4. CSQ4: Threat Coverage*

Tables 18 and 19 map each OWASP IoT Top 10 item to the methodology stage(s) mandating the countermeasure, the protocol phase(s) that provide the implementation-level evidence (cross-referenced with the phase analysis in Section 5.6 and Table 13), and the specific SMG mechanism.

Table 18: OWASP IoT Top 10 coverage analysis — Part 1 (I1–I5). Stage = methodology process mandating the countermeasure. Phase(s) = protocol phase(s) providing implementation evidence (see Table 13); — = addressed by policy/deployment, not protocol.

| # | OWASP IoT Risk | Stage(s) | Phase(s) | Evidence in SMG | Coverage |
|---|---|---|---|---|---|
| I1 | Weak/ hardcoded passwords | 2.2.1 | P1, P2 | No password or key is used for device identity: reconstructed at boot from a PUF using only non-secret mask/ECC helper data, never a stored password; confirmed working in 30/30 real trials (Section 6.3). The separate WiFi credential is provisioned into NVS out-of-band (`provision_server_ref.py`), not compiled into firmware source or the public binary; it remains one passphrase shared by every unit on the fleet's WPA2-PSK network, which is inherent to WPA2-PSK rather than a firmware defect (see Limitations) | Full |

*continued on next page*

*Table 18 continued from previous page*

| # | OWASP IoT Risk | Stage(s) | Phase(s) | Evidence in SMG | Coverage |
|---|---|---|---|---|---|
| I2 | Insecure network services | 2.2.4 | P2, P3 | P2 establishes mutual identification and a session key before any data is accepted; P3 encrypts every message with per-message random 12-byte nonce (AES-256-GCM); no plaintext channel exists | Full |
| I3 | Insecure ecosystem interfaces | 3.2.1 | P2 | P2 nonce-bound mutual identification, both parties verified before any session key is derived; node vs. user/admin access separated by identification channel (STRIDE EoP, Section 5.6); trust scoring exists but not yet ported to the RoT server (Limitations) | Full |
| I4 | Lack of secure update | 7.2.2 | — | OTA update policy (Stage 7.2.2): admin authentication + firmware signature required; implementation scoped to ITER 2 (policy artifact only, no signed-OTA deployment at TRL 3) | Partial |
| I5 | Insecure/ outdated compo-nents | 4.2 | — | ESP-IDF v5.4.2 pinned; no deprecated crypto primitives (ECDSA/SHA-256; AES-256-GCM via `mbedtls`) | Full |

Table 19: OWASP IoT Top 10 coverage analysis (continued from Table 18) — Part 2 (I6–I10).

| # | OWASP IoT Risk | Stage(s) | Phase(s) | Evidence in SMG | Coverage |
|---|---|---|---|---|---|
| I6 | Insufficient privacy | 1.4.3 + 2.2.4 | P3 | Data sovereignty policy (Stage 1.4.3): self-hosted RPi4; no PII; sensor data only; P3 encrypts all telemetry in transit with authenticated encryption | Full |
| I7 | Insecure data transfer/storage | 3.1.1 + 2.2.4 | P3 | P3 provides AES-256-GCM encryption in transit, now with forward secrecy (ephemeral ECDH, Table 3; hardware-validated, Section 6.3); no device-identity secret at rest. $V_B$ and the WiFi credential are both unencrypted in flash (standard ESP32 practice; flash encryption is an ITER 2 candidate) | Full |
| I8 | Lack of device management | 7.1 | — | Stage 7.1 server monitors heap, send rate, and session status from server-side logs (node_id, timestamps); without a cryptographic chain, this is an operational log rather than an independently verifiable activity audit | Partial |

*continued on next page*

*Table 19 continued from previous page*

| # | OWASP IoT Risk | Stage(s) | Phase(s) | Evidence in SMG | Coverage |
|---|---|---|---|---|---|
| I9 | Insecure default settings | 2.2.3 + 6.1.3 | P1 | No WiFi/server comms. until enrollment data exists in NVS; identity proof derives from the chip's physical PUF response, not a stored/default value; confirmed by cloning SMG_NODE_01's entire NVS onto SMG_NODE_02's physical chip, which the server correctly rejected (FT-06) | Full |
| I10 | Lack of physical hardening | — | — | Out of scope for software methodology; addressed at org. level (Stage 1.4.1); JTAG disable and secure boot specified for TRL 5+ | Partial |
| | **Coverage score (unweighted)** | | | | **7/10 full + 3/10 partial** |

**Answer to CSQ4:** Unweighted OWASP IoT Top 10 coverage of 7/10 items fully addressed and 3/10 partially addressed. Six of the seven fully-covered items (I1–I3, I6–I7, I9) have direct evidence from protocol phases P1–P3; the remaining one (I5: component vetting) is addressed by Stage 4 policy artifacts. The three partial items are I4 (secure update: policy defined, signed-OTA deferred to ITER 2), I8 (device management: server-side logging without a verifiable chain, a trade-off for a smaller footprint—see Limitations), and I10 (physical hardening: outside a software methodology's scope). The coverage score is an unweighted count, not a severity-weighted metric (B12, Section 6.5).

### *6.5. Expert Validation*

A structured questionnaire (14 Likert items, B1–B14, 5-point scale from 1 = strongly disagree to 5 = strongly agree, plus 3 open questions) was distributed to domain experts to supplement the case study evidence with inde-

pendent professional judgment. Fifteen evaluators completed the questionnaire; fourteen consented to their responses being used as validation evidence (one declined consent and was excluded prior to analysis), exceeding the upper bound of the planned 6–10 evaluator target. Six requested anonymity and are referenced anonymously below. Evaluators held primary expertise in cybersecurity, IoT systems design/development, smart-grid/energy management, software engineering, or embedded systems, with 1–15 years of professional or research experience and academic backgrounds ranging from B.Sc./B.Eng. to Ph.D., from institutions in Mexico, Spain, Portugal, and Canada. Eight of the fourteen are affiliated with institutions unconnected to the author team (CINVESTAV, ESCOM-IPN, a national Instituto Tecnológico, ECOSUR, Universidad Iberoamericana, Universidade de Aveiro, University of New Brunswick), making the panel majority-independent; the remaining six are affiliated with the author institutions (INAOE, home institution; CEIT-BRTA, co-author affiliation).

**Quantitative results:** excluding B6, B7, B9, B10 (superseded-design items, below), the headline mean was 3.94/5 (SD 1.02, median 4.0, 140 responses); across all 14 items, 3.99/5 (SD 0.96). Item means ranged from 3.57 (B12) and 3.71 (B1) to 4.43 (B10) and 4.36 (B3, B14); none fell below the questionnaire's neutral midpoint. B8 (dynamic RoT execution is verifiable, 4.00/5) corresponds directly to CSQ3. The independent-evaluator subset (N=8) returned a 10-item mean of 3.88/5 (14-item: 3.89/5), modestly below the affiliated subset's 4.02/5 (14-item: 4.11/5); this mild affiliation skew is mitigated by the panel's majority-independent composition.

**Qualitative findings that qualify the results:** B12 (OWASP coverage sufficiency) returned the lowest item mean (3.57/5): two evaluators noted the 9/10 OWASP score (as presented) is an unweighted count, not severity-weighted, and does not establish that I10 (physical hardening) is the least critical omission; subsequent reclassification of I4 and I8 to partial yields the revised 7/10 full + 3/10 partial reported in CSQ4 (Section 6.4). B1 (structure sufficiency) returned the second-lowest mean (3.71/5): three evaluators scored it 2/5, noting the methodology specifies *what* each stage must produce but not *how*—per-artifact templates or acceptance criteria would close this gap, echoed by several open-question (C3) responses. One evaluator qualified B6, noting the parameter count is an artifact of the SMG's four entity types rather than a fixed model limit (14 parameters, Table 12), with generalization to other entity types untested. Two evaluators gave B13 (cross-domain generalizability) dissenting scores (2/5, 1/5), treating it as an

Table 20: Expert questionnaire Likert results (N=14; 1 = strongly disagree, 5 = strongly agree). SD is the population standard deviation across the 14 responses to that item.

| Item | Statement (abbreviated) | Mean | SD | Mdn |
|---|---|---|---|---|
| B1 | Four-phase, seven-stage structure gives sufficient guidance without external references | 3.71 | 1.03 | 4.0 |
| B2 | Organizational phase (Stage 1) adds practical value ahead of architecture/technology decisions | 3.79 | 1.26 | 4.0 |
| B3 | Stage-by-stage artifact requirements make correct methodology application verifiable | 4.36 | 0.72 | 4.5 |
| B4 | TRL-guided iterative model with M1–M3 maintenance points is practical | 3.86 | 0.83 | 4.0 |
| B5 | Entity-based identity model is clearer than role/policy-based approaches | 3.86 | 0.99 | 4.0 |
| B6† | 12-parameter identity model generalizes beyond the SMG case study | 3.86 | 0.74 | 4.0 |
| B7† | RoT-as-process abstraction accommodates heterogeneous device capabilities | 4.29 | 0.80 | 4.5 |
| B8 | Dynamic RoT execution (Process 6.1.3) is a verifiable identity mechanism | 4.00 | 0.93 | 4.0 |
| B9† | Six-phase authentication/data-exchange protocol (P1–P6) is logically complete | 3.86 | 0.64 | 4.0 |
| B10† | Hash-chain mechanism (P4/P6) gives meaningful, feasible integrity guarantees | 4.43 | 0.62 | 4.5 |
| B11 | Technology-independent STRIDE mapping is more useful than technology-specific models | 4.07 | 1.16 | 4.5 |
| B12 | 90% OWASP IoT Top 10 coverage is sufficient security-by-design evidence at TRL 3 | 3.57 | 0.98 | 4.0 |
| B13 | Parameterized, technology-agnostic design applies beyond energy management | 3.79 | 1.15 | 4.0 |
| B14 | More systematic security integration than existing IoT methodologies | 4.36 | 0.72 | 4.5 |
| | **Full-panel mean (14 items, 196 responses)** | **3.99** | 0.96 | 4.0 |
| | **Headline mean, excl. † (10 items, 140 responses)** | **3.94** | 1.02 | 4.0 |

*Note († B6, B7, B9, B10): these item texts, as presented to evaluators, describe a 12-parameter identity model and a six-phase P1–P6 protocol that includes a hash-chain integrity mechanism. The current implementation (Section 6.3) uses a 14-parameter model, phases P1–P3, and no hash chain. The panel was not re-surveyed against the current implementation, so these four items are excluded from the headline mean reported elsewhere in this paper; the full-panel mean (including them) is reported here for completeness.*

*Note (B12): item text reflects OWASP coverage as presented to evaluators (9/10 full + 1/10 partial = 90%); the coverage assessment reported in CSQ4 (Section 6) is 7/10 full + 3/10 partial, since `I4` (secure-update policy) and `I8` (device management) are both classified as partial rather than full.*

unvalidated hypothesis (reflected in External Validity and Limitations below). One evaluator questioned whether same-team case-study evidence is sufficient without an independently-managed development team; the SMG implementation was carried out by multiple co-authors in distinct roles (Author Contributions), which partially addresses this. Two evaluators outside cybersecurity specifically (software-engineering methodology and general IoT systems) noted several protocol/threat-model items fell outside their expertise; their ratings are retained but should be weighted accordingly. The panel's majority-independent composition and subset means are reported above; the residual affiliation skew, convenience-sampling constraint, and preliminary-data caveat are acknowledged as limitations below.

## 7. Discussion

This work addresses two barriers to secure IoT ecosystem development: the security activation gap at the operational level, and the absence of a structured development methodology that integrates cybersecurity from the earliest organizational stage. The results of the confirmatory case study and expert validation are discussed below in relation to each research question.

**CSQ1 — Methodology guidance:** The stage-by-stage traceability (Tables 10 and 11) demonstrates that the methodology produces actionable decision points at every stage. The organizational phase (Stage 1) carries the highest leverage: classifying the SMG as critical infrastructure (Process 1.4.1) directly constrained the authentication scheme (requiring mutual cryptographic authentication), the storage approach (requiring isolation of identity material from firmware, met at TRL 3 by the NVS-partition/PUF design; a dedicated secure element is an ITER 2 requirement, see Limitations), and the access control granularity (requiring node, user, and admin distinction). Absent an explicit organizational stage, these constraints might not have been systematically propagated to architectural and implementation decisions.

**CSQ2 — Identity model:** The formal separation of identity into $P_{pub}$, $P_{priv}$, and $P_{auth}$ subsets yielded a technology-agnostic security specification that translated directly to ESP32 NVS storage for the PUF helper data and to an HKDF-derived ECDSA key pair for the private, non-persistent PUF response; `id_a`'s translation remains incomplete at TRL 3 (a compile-time placeholder rather than a provisioned per-device value, see Limitations). The 14-parameter model produced zero ambiguity across 4 entity types, confirm-

ing the model's expressiveness for heterogeneous IoT ecosystems, including resource-constrained devices.

**CSQ3 — Root of Trust verifiability:** The P2+P3 mutual authentication handshake, including the ephemeral-ECDH forward-secrecy and helper-data-manipulation fixes, completed in a median 1300 ms (mean $1313 \pm 48$ ms, split across server authentication, ECDSA sign/verify, and ECDH key derivation, Section 6.3) across $N = 30$ real hard-reset trials on SMG_NODE_01, with 30/30 trials succeeding end-to-end on the first attempt (Table 17). The single P3 demo message sent after each handshake was delivered and decrypted successfully in all 30 trials, and the device's derived key pair authenticated correctly against the server's registered public key in every trial, confirming a stable identity despite per-attempt PUF measurement noise.

**CSQ4 — Threat coverage:** The unweighted OWASP IoT Top 10 coverage (7/10 full; 3/10 partial: I4 secure-update policy only, I8 device management without a verifiable chain, I10 physical hardening) indicates the methodology systematically addresses known IoT threat categories. The STRIDE analysis operates at two levels: the methodology-level mapping (Tables 5/6, Section 4.7) shows every threat addressed by a mechanism mandated by at least one stage; the protocol-level mapping (Tables 13–14, Section 5.6) traces each mechanism to its implementing phase. All six categories are addressed, four by multiple independent phases. The three partial OWASP items reflect two deliberate trade-offs (I4, I8—see Limitations) and one inherent scope boundary (I10); the coverage figure is unweighted, not severity-weighted.

**Maintenance cycles:** At least six maintenance moments were triggered during Stages 5–6 (M2 firings for code-level revision, Section 6.3, Limitations), confirming the methodology's M2 pathway operates as specified at TRL 3. Each was caught during deployment-phase bench testing or subsequent review, before or alongside the reported trial; the case study does not establish a counterfactual about whether any would have surfaced only later absent that structure, since bench testing of this kind is standard practice regardless. M1 and M3 were not triggered and remain empirically unvalidated, available for future maintenance points and the operation-phase TRL transition.

### *7.1. Limitations*

- **Single topology:** Only centralized topology was implemented at TRL 3. Decentralized and mesh topologies supported by Stage 2.1.4 remain to be validated empirically.

- **TRL 3 scope:** The proof of concept operates in a controlled laboratory environment. Environmental variation, scalability beyond 2 nodes, and production-grade hardening (JTAG disable, secure boot) are deferred to ITER 2.

- **Primary-agent power dependency:** In the TRL 3 bench configuration the RPi4 primary agent is powered from the same DC bus it coordinates (via a DC/AC inverter); SSR duty transients and node resets cause primary-agent brownouts that can terminate sustained bench trials early. This coupling constrains sustained concurrent operation and is an artifact of the laboratory power arrangement rather than the methodology; ITER 2 will use an independent primary-agent supply.

- **Forward secrecy (ephemeral ECDH added, hardware-validated):** An earlier revision derived $Key_{ab}$ from $V_B$ and the handshake nonces alone, so recovering $V_B$ (e.g., from a flash dump, Table 6) together with a recorded handshake sufficed to recompute that session's key. $Key_{ab}$ now additionally depends on $Z$, a per-session ECDH secret discarded immediately after use, closing that gap: recovering $V_B$ alone no longer yields $Key_{ab}$ for any session. The fix is implemented (`rot_session.c`/ `rot_challenge_service.py`), passes the host-side protocol test suite (including a dedicated test that two handshakes against the same node yield different keys), and was validated end-to-end on the physical ESP32 against the real RPi4 server with a refreshed $N = 30$ bench round (30/30 success, Table 17, superseding the earlier pre-fix timing retained in the artifact repository for provenance).

- **Device-side PUF entropy:** The server stores only $pk_A$ and $V_B$ (Section 4), neither of which recreates $sk_A$ if disclosed; however, $sk_A$'s security still depends on the actual (not merely designed) entropy of the PUF response it derives from. Independent enrollments on one SMG node have measured 421–422-byte (∼3368–3376-bit) stable responses, somewhat above the library's ∼380–400-byte reference-hardware estimate (chip-to-chip and enrollment-to-enrollment variation, discussed below). Raw length is not min-entropy: repetition-code helper data is known to leak some response information (Dodis et al., 2004; Maes, 2013; Delvaux et al., 2015), so 3368 raw bits do not imply 256 bits of adversarial min-entropy **[TODO: min-entropy assessment pending]**: a dedicated

assessment (e.g., NIST SP 800-90B) remains the direct way to confirm sufficient min-entropy for the 256-bit scalar it seeds.

- **Raw PUF measurement left in NVS (found and fixed in a journal review pass, 2026-10-15):** `esp32_puflib`'s enrollment procedure (`provision_puf_calculate()`) wrote a `PUF_FREQUENCY` blob to NVS — the per-bit frequency count across all 20 enrollment measurements — and never erased it, contrary to this paper's claim that no raw PUF response is ever written to flash: that blob is sufficient to reconstruct the stable-bit mask and PUF reference without the physical device. The fix overwrites it with zeros and erases it (`nvs_erase_key()`) once the stable-bit mask and ECC helper data have been derived from it; unlike the mitigation below, this does not change the derived key, so it did not by itself require re-enrolling an already-provisioned device. This does not fully guarantee physical removal, and real hardware confirmed a more specific reason why: ESP-IDF's NVS is a log-structured store that alternates between two versions of a blob on every write, so the zero-overwrite only scrubs the *most recent* version — a raw dump taken immediately after a standard enrollment (before any further provisioning) showed the complete, un-zeroed 20-measurement table from an earlier version still present and byte-recoverable, logically marked erased but not yet overwritten (NVS only reclaims that space when it later garbage-collects the page). SMG_NODE_01's NVS as currently deployed carries none of this residue (confirmed directly: zero `PUF_FREQUENCY` entries of any kind in a fresh read-only dump), because the out-of-band step that pins the server reference happens to rebuild the NVS partition from scratch rather than writing incrementally; this is a side effect of that provisioning step, not a property the library itself guarantees, so the residue described above should still be assumed present on any node enrolled through the standard path alone. Only flash encryption (ITER 2 candidate, already noted above) closes this fully regardless of provisioning method.

  Separately, `PUF_MASK`/`ECC_DATA` (and their deep-sleep-retry counterparts, `PUF_SLEEP_MASK`/`ECC_SLEEP_DATA`) are legitimate public helper data under standard fuzzy-extractor constructions (the device's 8×-repetition code is a code-offset secure sketch), but they are stored in NVS without integrity protection; an attacker with physical write access to this same chip's flash could manipulate them and use the ECC decoder's error count (surfaced via diagnostic logging, needed for this work's own bench

measurements, Section 6.3) as an oracle on individual response bits. The key derivation binds SHA-256 of all four helper-data blobs into the HKDF info used to derive $(sk_A, pk_A)$ (initially only the first two, later extended to all four after a second review pass found the retry-path pair was left unbound), so manipulating any of them changes the derived key unpredictably instead of the key itself leaking information; a direct single-bit-flip test on each of the two retry-path blobs confirmed this for the extended binding (Section 6.3). This is a narrower guarantee than "closes the oracle": the error-count value is computed and logged *before* the HKDF binding is applied, so an attacker able to observe that diagnostic output (not just the handshake's success or failure) over UART still learns something about which bits were manipulated, independent of what key eventually gets derived from them; a production deployment should compile that diagnostic out, which this TRL 3 validation firmware deliberately does not do. The binding also does not substitute for secure boot or flash encryption (ITER 2 candidates, not yet enabled): an attacker with write access to this chip's flash could instead replace the firmware itself with one that reads and transmits the reconstructed response directly, bypassing the derivation entirely. Because the binding changes the derived key, fixing it required re-enrolling SMG_NODE_01 and re-registering the resulting $pk_A$ with the server on each of the two occasions the binding's scope changed; re-validated end-to-end both times: $N = 30$ hard-reset trials reached 30/30 success, all on the first attempt (Table 17), and the cross-chip cloning test (FT-06) was repeated against each new identity with the same result as before (rejected by the server in every attempt).

- **PUF library build and bench status (validated on one of two nodes):** `esp32_puflib` (Stanicek, 2022) is a native C/ESP-IDF component and is linked directly into the node's C/ESP-IDF firmware (`smg_puf_rot_node`), without a language-runtime wrapper. The identity/Root-of-Trust/secure-communication firmware and its RPi4 counterpart (`rot_challenge_service.py`) build cleanly, and the firmware has been flashed to, enrolled on, and bench-validated end-to-end against one of the SMG's two physical ESP32 units: $N = 30$ real hard-reset trials against the real RPi4 server reached 30/30 end-to-end success (Section 6.3). SMG_NODE_02 has not yet been enrolled under this scheme in its own right, so two-node reconstruction-reliability validation (Table 17-style bench statistics for a second chip) remains open.

Physical distinctness across chips, however, was directly confirmed by cloning SMG_NODE_01's entire NVS onto SMG_NODE_02's hardware: applying SMG_NODE_01's helper data to SMG_NODE_02's physical RTC-SRAM failed the library's original raw-bit-error-rate threshold on the first attempt (29.9% against a 0.15% requirement) but passed it on the retry, since that threshold was not yet understood to be a non-security diagnostic at the time this test was first run (it has since been relaxed, Section 6.3). The test was repeated three times more as the firmware was further corrected, to check that none of the fixes changed the outcome: first under the error-rate-relaxed firmware (4 separate reset attempts), where the device now locally accepts the cross-chip reconstruction on the first attempt (29.8% error, no retry triggered) but the server rejected the resulting signature in all 4 attempts; then again after SMG_NODE_01's re-enrollment under the first-generation helper-data-manipulation mitigation (3 separate reset attempts, ∼29.9% error each), with the same result in all 3; and once more after the mitigation was extended to bind all four helper-data blobs, requiring a further re-enrollment (3 separate reset attempts, 29.77–29.79% error each), again rejected in all 3. Detection rests entirely on the server's ECDSA verification, not any device-side check (FT-06, `c4_negative_test/`), which is exactly why relaxing the device-side diagnostic does not weaken this guarantee – it only changes when, not whether, the wrong reconstruction is caught. The current firmware also uses a compile-time placeholder node identifier shared by all units; it identifies the node on the wire but does not enter the authentication proof, and is to be replaced by per-unit assignment before multi-node deployment.

- **Sensor/EMS modules not yet implemented (C/ESP-IDF firmware scope):** The `smg_puf_rot_node` C/ESP-IDF firmware currently implements only the identity (PUF), Root of Trust, and secure-communication layers; fuzzy Energy Management System and sensor-acquisition logic have not yet been implemented in this firmware. A full end-to-end SMG deployment requires this additional work, distinct from and in addition to the hardware bench validation above. The same gap applies to the server side: the primary agent's anomaly-based trust scoring and quarantine logic (Stage 7.2.1) exists in `smg_primary_server`'s original module set (`auth_service.py`, `ems_coordinator.py`) but has not yet been ported to `rot_challenge_service.py`, the server counterpart

of the current RoT protocol; DoS/I3 coverage claims elsewhere in this paper that cite trust scoring describe the methodology-mandated mechanism and its prior instantiation, not a property of the currently-deployed RoT server.

- **No cryptographic session-chain audit (deliberate design simplification):** The SMG's Root of Trust does not implement a per-message hash chain to provide a retrospectively-verifiable, third-party-auditable record of session integrity. Message-level integrity is provided by the AES-256-GCM authentication tag (P3), but there is no cryptographic chain binding messages within a session, and Repudiation/I8 (device management) coverage is therefore evidentiary (server-side session logs) rather than independently verifiable (Table 5, Table 19). Introducing a lightweight, chain-free sequencing mechanism (e.g., a monotonic per-session counter bound into the GCM associated data) without the overhead of a full chain audit is a candidate ITER 2 refinement if reviewers or subsequent evaluation identify this trade-off as unacceptable.

- **PUF reliability on the deployed hardware (reconstruction reliability measured on one node; thermal/second-node characterization pending):** `esp32_puflib`'s enrollment procedure targets a design bit-flip probability of 0.001–0.005 via a stable-bit mask and 8×-repetition-code error correction, yielding an estimated ∼380–400-byte stable response in the library's own design notes for its reference hardware. Independent enrollment runs on SMG_NODE_01 have produced stable responses of 421–422 bytes — consistently reproducible, but somewhat above that figure, a plausible chip-to-chip variation rather than a conformance failure, since the library documents its figure as an estimate, not a hard specification. Reconstruction reliability was measured over six $N = 30$ hard-reset bench rounds across the project's history (Table 17, Section 6.3). The library's own reliability gate originally combined a hamming-weight sanity check with a raw bit-error-rate threshold (0.15%); the latter turned out not to be a security boundary — the ECC's majority-vote decode runs regardless of the error count, and a wrong reconstruction fails the server's ECDSA verification downstream exactly like any other incorrect answer — so it was relaxed to a diagnostic during this work, after an intermediate round run under the original, unrelaxed gate rejected every single trial (first-attempt error 0.455–0.760%, retry-path er-

ror 0.226–0.406%, both above the 0.15% figure). The round reported here, collected after that fix and the subsequent helper-data-manipulation mitigation, shows no bimodal attempt pattern: every trial succeeds on the first attempt, and the earlier two-attempt pattern seen in the 2026-10-13 round is now understood as an artifact of that gate, not a genuine circuit-level difference between the library's two reset depths. The 2026-10-13 round's attempt-1 error rate also rose measurably across its own ∼5-minute duration, suggesting within-run thermal drift contributes alongside enrollment-to-enrollment differences; the round reported here was collected over a shorter span and does not show the same trend as clearly. What remains unmeasured is (a) sensitivity to temperature and supply-voltage variation in a controlled way, and (b) whether the same response length and reliability hold on SMG_NODE_02 and across further enrollments **[TODO: temperature/voltage and cross-node data pending]**: this remains pending before the reliability claim extends beyond SMG_NODE_01 under laboratory-ambient conditions.

- **Single application domain:** Energy management is the only domain tested. The expert questionnaire (N=14, Section 6.5) supports the technology-agnostic design intent (B13 mean 3.79/5) but also recorded two dissenting scores on this same item (2/5 and 1/5), with the dissenting evaluators characterizing cross-domain generalizability as an unvalidated hypothesis; empirical validation in additional domains remains future work.

- **Physical hardening:** I10 (physical hardening) and hardware-based Root of Trust (e.g., ATECC608 secure element) are not addressed at TRL 3 but are specified as ITER 2 requirements.

- **Identity lifecycle (methodology gap):** The methodology as presented does not specify a complete identity lifecycle process (parameter rotation, revocation, recovery after compromise without full re-provisioning, ownership transfer, and decommissioning). This is a gap in the methodology itself—not solely a TRL 3 scope limitation—since recovery from a compromised node currently requires re-running the full Root of Trust. An expert evaluator (Section 6.5) identified this as the single addition with the greatest potential impact on practical applicability; it is the highest-priority extension for the next methodology revision and is deferred to ITER 2.

- **User/administrator password hashing:** Users and administrators authenticate via a plain SHA-256 hash of their password (Table 12), not a salted, slow key-derivation function (bcrypt, scrypt, Argon2) — weaker than current practice, and a candidate ITER 2 hardening item; it does not affect the PUF-based IoT node scheme, which uses no password.

- **WiFi credential provisioning (fixed; fleet-sharing residual):** An earlier firmware revision compiled the WiFi SSID/password into source as a fixed constant, identical for every build and visible in the published firmware source. The credential is now provisioned into NVS out-of-band by `tools/provision_server_ref.py` (the same mechanism already used for $V_B$), so no WiFi credential value appears in firmware source or the compiled binary. One property is unchanged by this fix and is not a firmware defect: every node on the fleet's WPA2-PSK access point necessarily shares the same passphrase, so recovering one node's NVS still discloses the credential needed to join that network, same as recovering any WPA2-PSK passphrase would. Rotating it still requires re-provisioning every node's NVS image, not merely one.

- **Per-device node identifier not yet provisioned:** `id_a`, the device's wire identifier and server-registry key, is currently a compile-time placeholder shared by every unit built from the same firmware source (`rot_identity.c`), not a value generated and written to NVS per device at enrollment. This is an unresolved gap between the identity model's specification (Table 12) and the TRL 3 implementation: a second physical node cannot be registered under this scheme without first assigning it a distinct `id_a`, since the server registry is keyed on it. `id_a` is not itself part of the cryptographic authentication proof (that rests entirely on the PUF-derived key pair), so this gap does not weaken device authentication, but it does mean the single-node bench validation reported here (Section 6.3) cannot be read as demonstrating multi-node identity distinctness through `id_a`; FT-06's cross-chip test (Section 6.3) demonstrates that distinctness through the PUF response instead. Provisioning a real per-device `id_a` is a prerequisite for validating SMG_NODE_02 under this scheme and is deferred to ITER 2.

- **Expert validation:** Fourteen evaluators completed the questionnaire (14 Likert items, 3 open questions), exceeding the planned 6–10 target, yielding a headline agreement mean of 3.94/5 (10 items excluding 4 that

describe the superseded design; 3.99/5 across all 14, Section 6.5). Eight of the fourteen are affiliated with institutions unconnected to the author team (majority-independent), though recruitment remained convenience-sampled; the independent subset (3.88/5) rated modestly below the affiliated subset (4.02/5), a mild skew the panel composition mitigates. At least two affiliated evaluators (Acknowledgements) are MSc students in the authors' own department, a power relationship beyond simple affiliation this study does not separately quantify. The condensed reference document distributed to evaluators (June 24, 2026) predates the final bench trials and used more conservative figures. One additional respondent declined consent and was excluded.

### *7.2. Threats to Validity*

Table 21: Threats to validity.

| **Type** | **Specific Threat** | **Mitigation** |
|---|---|---|
| Internal validity | SMG decisions may reflect author preference rather than methodology guidance; the traceability tables are an authored narrative and cannot fully rule this out on their own | Stage-by-stage traceability (Tables 10 and 11) documents an explicit process → decision chain; an independent code-level audit (AI-assistance declaration) cross-checked specific claims against the firmware/server source and forced correction of several that did not hold (e.g. the device-identifier and session-key claims in Limitations) — evidence against purely self-serving narrative, though not a substitute for independent replication |
| External validity | Single domain (energy) + single topology (centralized) limits generalizability | Methodology is technology-agnostic by design; expert questionnaire (N=14, headline mean 3.94/5, Section 6.5) supports this design intent, though three evaluators noted that generalization beyond the validated domain and entity set remains an untested hypothesis |
| Construct validity | TRL 3 does not demonstrate production readiness | TRL 3 is the stated objective of ITER 1; the methodology explicitly supports iterative maturation toward TRL 5+ |
| Reliability | Physical prototype measurements subject to environmental variation | $N = 30$ real hard-reset trials on SMG_NODE_01 (Section 6.3): handshake latency std 48 ms, end-to-end success 30/30; single-node scope and ambient laboratory conditions are disclosed limitations |

## 8. Conclusions

This work has presented a security-by-design methodology for centralized IoT ecosystems that addresses the security activation gap by integrating cybersecurity systematically from the earliest organizational stage. The methodology's four phases and seven stages produce documented, traceable artifacts; the entity-centric identity model provides technology-agnostic security specifications applicable to heterogeneous devices; and the dynamic Root of Trust process establishes verifiable cryptographic identity before any operational communication begins.

Empirical validation through a confirmatory case study (Yin, 2014) applying the methodology to a DC Smart Microgrid IoT ecosystem yields four findings. First, all seven stages produced complete, traceable outputs with zero external guidance required (CSQ1). Second, the 14-parameter identity model across four entity types produced zero ambiguity (CSQ2). Third, the RoT process established verifiable mutual authentication: a PUF-based scheme (`esp32_puflib` (Stanicek, 2022)) that reconstructs each node's identity from non-persistent physical characteristics rather than a stored key, combined with an asymmetric (ECDSA) device-authentication proof, a per-node symmetric server-authentication proof, and an ephemeral ECDH exchange giving the session key forward secrecy, validated end-to-end on real hardware for one of the two nodes across $N = 30$ hard-reset trials (30/30 success, every trial on the first attempt, median handshake latency 1300 ms, mean $1313 \pm 48$ ms); this validation also found and fixed a miscalibrated local reliability gate that had originally rejected every attempt outright, a raw PUF measurement left unerased in NVS, and a helper-data-manipulation oracle that an earlier form of the mitigation did not fully close (Limitations); only the last of these changes the derived key and required re-enrolling the device, and validating SMG_NODE_02 remains future work (CSQ3, Section 6.3). Fourth, the methodology achieved unweighted OWASP IoT Top 10 coverage of 7/10 items fully addressed and 3/10 partially addressed (I4 secure-update policy only; I8 device management, a trade-off for a smaller on-device footprint; I10 physical hardening outside software scope) (CSQ4). This case study evidence is supplemented by an expert questionnaire (N=14, Section 6.5) that returned a headline agreement mean of 3.94/5 (10 items; 3.99/5 across all 14), with the strongest agreement on methodology completeness and systematic security integration (B3, B14 means 4.36/5) and qualified agreement on cross-domain generalizability and threat-coverage weighting.

The methodology's parameterized, technology-agnostic structure is designed to support application across diverse IoT domains; cross-domain generalization is a design intent supported by expert opinion (B13 mean 3.79/5, with dissenting 2/5 and 1/5 ratings characterizing it as an unvalidated hypothesis) rather than a demonstrated property, and empirical validation in additional domains remains future work. Future work will extend validation to: ITER 2 (TRL 5+ with hardware security elements and signed-OTA implementation), cross-domain deployment (healthcare, industrial automation), an expanded expert evaluator pool with improved institutional independence, formal cryptographic protocol verification of the SMG's PUF-based authentication scheme, resolution of the identity-lifecycle gap (rotation, revocation, recovery), evaluation of per-node verifier wrapping or HSM-backed server storage to reduce central-server single-point-of-failure risk, and a lightweight chain-free sequencing mechanism to recover stronger Repudiation/I8 coverage without the on-device footprint of a full cryptographic session-chain audit.

## Declarations

*Ethics Statement*

The expert-validation questionnaire (Section 6.5) collected professional opinions from domain-expert volunteers; it did not collect sensitive personal data, and participation required explicit informed consent (item D1) covering use of responses as validation evidence, with an option for anonymous participation and the right to decline (exercised by one respondent, excluded prior to analysis). **[TODO: No formal institutional ethics-committee approval or exemption was obtained for this questionnaire; the authors should confirm whether their institution requires one for this kind of professional-opinion survey and add the relevant approval/exemption reference here if so.]**

*Materials Availability*

The ESP32 firmware (`smg_puf_rot_node/`) and primary agent software (`smg_primary_server/`) are available under Apache 2.0 license in the companion code repository (Campos-Sanchez et al., 2026b): `https://github.com/RazielCS/smg-secure-microgrid`. Within `smg_primary_server/`, only `puf_server.py` and `rot_challenge_service.py` ran during the reported bench trials; the directory also retains an earlier, superseded

module set (`auth_service.py`, `server.py`, `pc_client.py`, `protocol.py`, `crypto_utils.py`, `ems_coordinator.py`), kept because part of its functionality has not yet been ported (see Limitations) and clearly labeled as such in the repository's README. The complete research artifacts — manuscript source, case-study documentation, raw bench-trial measurements (canonical CSVs and serial logs under `case study SMG/Implementation/smg_puf_rot_node/`), and the expert-validation instruments — are available in the companion artifact repository (Campos-Sanchez et al., 2026c): https://github.com/RazielCS/smg-secure-microgrid-paper-artifacts. Device- and server-side secrets (the real PUF verifiers, NVS images, and WiFi credentials) are excluded from both repositories and replaced with sanitized `.example.json` templates and placeholder values; see each repository's README for the exact exclusion list.

*Data Statement*

The raw bench-trial data underlying this work's quantitative RoT findings — the $N = 30$ hard-reset trial's per-run serial logs and parsed summary (Table 17), the server-side log captured during that run, and the FT-06 cross-chip cloning negative test's device/server logs — have been deposited in, and are cited and linked from, the companion artifact repository (Campos-Sanchez et al., 2026c). The expert-validation instrument (questionnaire text) is deposited there as well; the individual evaluators' raw responses are not, since several evaluators requested anonymity (Acknowledgements) and consent was not obtained to publish raw response-level data. No packet captures, enrollment logs, or stage-artifact documents (risk register, access-control matrix, etc.) beyond what is reported in the manuscript and supplementary material were produced as separate deposit-ready files. The device/server secrets noted under Materials Availability are excluded for security rather than confidentiality reasons and are not needed to reproduce or verify any reported result.

*Competing Interests*

The authors declare no competing financial interests or personal relationships that could have appeared to influence this work.

*Funding*

R. Campos-Sanchez thanks SECIHTI for scholarship support (CVU 623274).

*Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process*

During the preparation of this work, the authors used Claude (Anthropic, Claude Code) as an AI-assisted software engineering and writing-support tool for: implementing, debugging, and bench-testing the case study's firmware and server software; statistical analysis of the real-hardware bench-trial data reported in Section 6.3; drafting and revising manuscript text, including cross-section consistency verification; and preparing the companion code and artifact repositories cited in Materials Availability. An independent AI coding assistant (OpenCode, running third-party models) was additionally used to perform an independent critical-review pass of the manuscript, whose findings the authors evaluated and selectively incorporated. After using these tools, the authors reviewed, verified, and edited all AI-assisted output and take full responsibility for the content of the published article. AI tools are not listed as authors or co-authors of this work.

*Acknowledgements*

The authors thank the following experts for completing the validation questionnaire (Section 6.5) and consenting to attribution: Jose de Jesus Zapata Lara (CINVESTAV); Jorge Andres Larracoechea, PhD (Asociacion Centro Tecnologico Ceit); Claudia García-Blanquel, PhD (ESCOM-IPN); Edwin Pacheco, MSc student in Security Sciences and Technologies (INAOE); Agustin Alberto Ahumada Ramirez, MSc student in Security Sciences and Technologies (INAOE); Yair Romero Lopez, M.Eng. (INAOE); Alfonso Martínez-Cruz, PhD (SECIHTI–INAOE); and Eder Toledo, MSc (ECOSUR). Six additional evaluators contributed responses under conditions of anonymity and are not named here per their request. One further respondent declined consent to the use of their responses and was excluded prior to analysis.

*Author Contributions*

**R. Campos-Sanchez:** Conceptualization, investigation, formal analysis, writing — original draft, implementation. **L. Hernandez-Martinez:** Supervision, validation, writing — review & editing. **C. Feregrino-Uribe:** Supervision, validation, writing — review & editing. **A. Penuelas-Angulo:** Conceptualization, formal analysis, writing — original draft. **D. Cruz-Aguilar:** Conceptualization, validation, implementation, writing — review & editing.

*Vitae*

**[TODO: author biographies and photographs pending]**

A short (maximum 100-word) biography and a passport-type photograph for each of the 5 authors — R. Campos-Sanchez, L. Hernandez-Martinez, C. Feregrino-Uribe, A. Penuelas-Angulo, D. Cruz-Aguilar — are to be supplied by each author before submission; photographs are to be submitted as separate figure files, not embedded here.

## References


Abraham, D., Houmb, S.H., Erdodi, L., 2025. Cyber-attacks on energy infrastructure—a literature overview and perspectives on the current situation. Applied Sciences 15, 9233.

Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., Ayyash, M., 2015. Internet of things: A survey on enabling technologies, protocols, and applications. IEEE Communications Surveys & Tutorials 17, 2347–2376.

Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., Cochran, J., Durumeric, Z., Halderman, J.A., Invernizzi, L., Kallitsis, M., Kumar, D., Lever, C., Ma, Z., Mason, J., Menscher, D., Seaman, C., Sullivan, N., Thomas, K., Zhou, Y., 2017. Understanding the Mirai botnet, in: Proceedings of the 26th USENIX Security Symposium, USENIX Association, Vancouver, BC, Canada. pp. 1093–1110.

Ayala, I., Amor, M., Fuentes, L., Troya, J.M., 2015. A software product line process to develop agents for the IoT. Sensors 15, 15640–15660.

Bakiris, E., Oikonomakis, G., 2025. IoT-driven security solutions, in: Cybersecurity: 9th European Interdisciplinary Cybersecurity Conference, EICC 2025, Rennes, France, June 18–19, 2025, Proceedings, Springer Nature. p. 275.

Blanco, C., Rosado, D.G., Varela-Vaca, Á.J., Gómez-López, M.T., Fernández-Medina, E., 2023. Onto-CARMEN: Ontology-driven approach for Cyber–Physical System security requirements meta-modelling and reasoning. Internet of Things 24, 100989.

Campos-Sanchez, R., Feregrino-Uribe, C., Hernandez-Martinez, L., Penuelas-Angulo, A., 2026a. An identity-centric paradigm for the internet of things: Security, trust, and sovereignty by design. IoT Submitted for publication.

Campos-Sanchez, R., Hernandez-Martinez, L., Feregrino-Uribe, C., Penuelas-Angulo, A., Cruz-Aguilar, D., 2026b. smg-secure-microgrid: Companion code repository for "closing the security activation gap". Software repository, https://github.com/RazielCS/smg-secure-microgrid.

Campos-Sanchez, R., Hernandez-Martinez, L., Feregrino-Uribe, C., Penuelas-Angulo, A., Cruz-Aguilar, D., 2026c. smg-secure-microgrid-paper-artifacts: Research artifacts and bench-trial data for "closing the security activation gap". Dataset repository, https://github.com/RazielCS/smg-secure-microgrid-paper-artifacts.

Casadei, R., Tsigkanos, C., Viroli, M., Dustdar, S., 2019. Engineering resilient collaborative edge-enabled IoT, in: 2019 IEEE International Conference on Services Computing (SCC), IEEE. pp. 36–45.

Cicirelli, F., Fortino, G., Guerrieri, A., Spezzano, G., Vinci, A., 2017. Metamodeling of smart environments: from design to implementation. Advanced Engineering Informatics 33, 274–284.

Cloud Security Alliance, 2023. IoT Security Controls Framework, Version 3. Technical Report. Cloud Security Alliance. URL: https://cloudsecurityalliance.org/research/working-groups/internet-of-things.

Costa, B., Pires, P.F., Delicato, F.C., 2016. Modeling IoT applications with SysML4IoT, in: 2016 42nd Euromicro Conference on Software Engineering and Advanced Applications (SEAA), IEEE. pp. 157–164.

Delvaux, J., Gu, D., Schellekens, D., Verbauwhede, I., 2015. Helper data algorithms for PUF-based key generation: Overview and analysis. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 34, 889–902.

Dodis, Y., Reyzin, L., Smith, A., 2004. Fuzzy extractors: How to generate strong keys from biometrics and other noisy data. Advances in Cryptology (EUROCRYPT 2004) , 523–540.

European Telecommunications Standards Institute, 2020. ETSI EN 303 645 V2.1.1: Cyber security for consumer internet of things: Baseline requirements. European Standard. URL: https://www.etsi.org/deliver/etsi_en/303600_303699/303645/02.01.01_60/en_303645v020101p.pdf.

European Union Agency for Cybersecurity, 2017. Baseline Security Recommendations for IoT in the Context of Critical Information Infrastructures. Technical Report. European Union Agency for Cybersecurity (ENISA).

Fagan, M., Marron, J., Brady, K., Cuthill, B., Megas, K., Herold, R., 2021. IoT Device Cybersecurity Guidance for the Federal Government: Establishing IoT Device Cybersecurity Requirements. NIST Special Publication 800-213. National Institute of Standards and Technology (NIST). Gaithersburg, MD. doi:10.6028/NIST.SP.800-213.

Fernandez, E.B., Brazhuk, A., 2024. A critical analysis of Zero Trust Architecture (ZTA). Computer Standards & Interfaces 89, 103832.

Fernando, Y., Tseng, M.L., Wahyuni-Td, I.S., de Sousa Jabbour, A.B.L., Chiappetta Jabbour, C.J., Foropon, C., 2023. Cyber supply chain risk management and performance in industry 4.0 era: information system security practices in Malaysia. Journal of Industrial and Production Engineering 40, 102–116.

Fortino, G., Russo, W., Savaglio, C., Shen, W., Zhou, M., 2017. Agent-oriented cooperative smart objects: From IoT system design to implementation. IEEE Transactions on Systems, Man, and Cybernetics: Systems 48, 1939–1956.

Fortino, G., Savaglio, C., Spezzano, G., Zhou, M., 2020. Internet of things as system of systems: A review of methodologies, frameworks, platforms, and tools. IEEE Transactions on Systems, Man, and Cybernetics: Systems 51, 223–236.

Guajardo, J., Kumar, S.S., Schrijen, G.J., Tuyls, P., 2007. FPGA intrinsic PUFs and their use for IP protection, in: Cryptographic Hardware and Embedded Systems (CHES 2007), Springer. pp. 63–80.

Guerrero-Ulloa, G., Rodriguez-Dominguez, C., Hornos, M.J., 2023. Agile methodologies applied to the development of Internet of Things (IoT)-based systems: A review. Sensors 23, 790.

ISO/IEC, 2019. ISO/IEC 9798-3:2019 information technology — security techniques — entity authentication — part 3: Mechanisms using digital signature techniques. International Organization for Standardization. 5th edition.

Javanmardi, S., Shojafar, M., Mohammadi, R., Alazab, M., Caruso, A.M., 2023. An SDN perspective IoT-fog security: A survey. Computer Networks 229, 109732.

Josyula, S.K., Gupta, D., 2017. A new security methodology for internet of things, in: 2017 International Conference on Computing, Communication and Automation (ICCCA), IEEE. pp. 613–618.

Licata, J., McWhite, R., Calloway, L., Gilbert, D., Anderson, M., Snyder, J., Miller, J., 2025. Developing Security, Privacy, and Cybersecurity Supply Chain Risk Management Plans for Systems. Technical Report. National Institute of Standards and Technology.

Maes, R., 2013. Physically Unclonable Functions: Constructions, Properties and Applications. Springer.

Makhdoom, I., Abolhasan, M., Lipman, J., Liu, R.P., Ni, W., 2018. Anatomy of threats to the internet of things. IEEE Communications Surveys & Tutorials 21, 1636–1675.

Manate, B., Forti, F., Moore, P., 2014. Applying the Prometheus methodology for an Internet of Things architecture, in: 2014 IEEE/ACM 7th International Conference on Utility and Cloud Computing, IEEE. pp. 435–442.

Merzouk, S., Cherkaoui, A., Marzak, A., Nawal, S., 2020. IoT methodologies: comparative study. Procedia Computer Science 175, 585–590.

Moreta, N., Aragon, D., Ona, S., Jaramillo, A., Ibarra, J., Jahankhani, H., 2023. Comparison of cybersecurity methodologies for the implementing of a secure IoT architecture, in: Cybersecurity in the Age of Smart Societies: Proceedings of the 14th International Conference on Global Security, Safety and Sustainability, London, September 2022, Springer. pp. 9–29.

Morin, B., Harrand, N., Fleurey, F., 2017. Model-based software engineering to tame the IoT jungle. IEEE Software 34, 30–36.

Muñoz, M., Torres, M., Gil, J.D., Guzmán, J.L., 2025. An Internet of Things platform for heterogeneous data integration: Methodology and application examples. Journal of Network and Computer Applications 240, 104197.

Okika, N., Nwatuzie, G.A., Odozor, L., Oni, O., Idoko, I.P., 2025. Addressing IoT-driven cybersecurity risks in critical infrastructure to safeguard public utilities and prevent large-scale service disruptions. International Journal of Innovative Science and Research Technology 10.

Patel, P., Cassou, D., 2015. Enabling high-level application development for the Internet of Things. Journal of Systems and Software 103, 62–84.

Ray, R.K., Chowdhury, F.R., Hasan, M.R., 2024. Blockchain applications in retail cybersecurity: Enhancing supply chain integrity, secure transactions, and data protection. Journal of Business and Management Studies 6, 206–214.

Saqib, M., Moon, A.H., 2023. A systematic security assessment and review of Internet of Things in the context of authentication. Computers & Security 125, 103053.

Sassalou, E.N., Vasileiadis, S., Kazazis, S.A., Protogerou, G., Varvitsiotis, N., Karas, D.S., Giannetsos, T., Tsintzos, S., 2025. A PUF-based root-of-trust for resource-constrained IoT devices, in: 2025 IEEE International Conference on Cyber Security and Resilience (CSR), IEEE. pp. 824–831.

Savaglio, C., Fortino, G., Gravina, R., Russo, W., 2018. A methodology for integrating internet of things platforms, in: 2018 IEEE International Conference on Cloud Engineering (IC2E), IEEE. pp. 317–322.

Shahinzadeh, G., Shahinzadeh, H., Tanwar, S., 2024. Security and privacy issues in the Internet of Things: A comprehensive survey of protocols, standards, and the revolutionary role of blockchain, in: 2024 8th International Conference on Smart Cities, Internet of Things and Applications (SCIoT), IEEE. pp. 59–67.

Shostack, A., 2014. Threat Modeling: Designing for Security. Wiley, Indianapolis, IN.

Sorri, K., Mustafee, N., Seppanen, M., 2022. Revisiting IoT definitions: A framework towards comprehensive use. Technological Forecasting and Social Change 179, 121623.

Spanoudakis, N., Moraitis, P., 2015. Engineering ambient intelligence systems using agent technology. IEEE Intelligent Systems 30, 60–67.

Stanicek, O., 2022. esp32_puflib: An SRAM-based physical unclonable function library for the ESP32 microcontroller family. Software library, ESP-IDF component, https://github.com/staniond/esp32_puflib.

Turner, S.W., Karakus, M., Guler, E., Uludag, S., 2023. A promising integration of SDN and blockchain for IoT networks: A survey. IEEE Access .

UK Government Security, 2024. Secure by design principles. https://www.security.gov.uk/guidance/secure-by-design/principles/. Accessed: 30 April 2024.

Visintin, A., 2024. Secure IoT and the Identity of Things. Ph.D. thesis. Università degli Studi di Padova.

Yin, R.K., 2014. Case Study Research: Design and Methods. 5th ed., SAGE Publications, Thousand Oaks, CA.

Zanasi, C., Russo, S., Colajanni, M., 2024. Flexible zero trust architecture for the cybersecurity of industrial IoT infrastructures. Ad Hoc Networks , 103414.

Zhou, I., Makhdoom, I., Shariati, N., Raza, M.A., Keshavarz, R., Lipman, J., Abolhasan, M., Jamalipour, A., 2021. Internet of things 2.0: Concepts, applications, and future directions. IEEE Access 9, 70961–71012.